

Interaktive Lernplattform für Grundlagen Informatik

Projektarbeit – SS 26

9. September 2026

Studenten:	Markus Nöth	markus.noeth@study.thws.de	5123027
	Richard Schröder	richard.schroeder@study.thws.de	5123042
	Enrico Ebert	enrico.ebert@study.thws.de	5123098
	Duy Tan Pham	duytan.pham@study.thws.de	5123063
	Kristian Popov	kristian.popov@study.thws.de	5123029

Dozent: Prof. Dr. Peter Braun

Inhaltsverzeichnis

1 Einleitung **1**
1.1 Problemstellung (Enrico Ebert) . . . 1
1.2 Zielsetzung der Arbeit (Duy Tan Pham) . . . 1
1.3 Methodik (Markus Nöth) . . . 2

2 Übergreifende Architektur & Systemdesign **2**
2.1 Technologiestack (Richard Schröder) . . . 2
2.2 Modulare Softwarearchitektur (Markus Nöth) . . . 3
2.3 Unified UI-Konzept (Duy Tan Pham) . . . 4
2.4 Simulator-Architektur (Richard Schröder) . . . 5
2.5 Datenmodellierung und Transfer (Richard Schröder) . . . 6

3 Einsatz von Künstlicher Intelligenz im Projekt **7**
3.1 KI-Reflexion (Markus Nöth) . . . 7
3.1.1 KI als Co-Developer (Markus Nöth) . . . 7
3.1.2 Aufgabengenerierung und manuelle Prüfung (Markus Nöth) . . . 8
3.1.3 Dokumentation und Markdown-Dateien (Markus Nöth) . . . 8
3.1.4 Kritische Reflexion (Markus Nöth) . . . 8
3.2 KI-Reflexion (Richard Schröder) . . . 8
3.2.1 KI als Co-Developer (Richard Schröder) . . . 8
3.2.2 Automatisierte Aufgabengenerierung (Richard Schröder) . . . 9
3.2.3 Kritische Reflexion (Richard Schröder) . . . 9
3.3 KI Reflexion (Enrico Ebert) . . . 10
3.3.1 KI als Co-Developer (Enrico Ebert) . . . 10
3.3.2 Kritische Reflexion (Enrico Ebert) . . . 10
3.4 KI Reflexion (Duy Tan Pham) . . . 11
3.4.1 KI als Co-Developer (Duy Tan Pham) . . . 11
3.4.2 Automatisierte Aufgabengenerierung (Duy Tan Pham) . . . 11
3.4.3 Kritische Reflexion (Duy Tan Pham) . . . 11
3.5 KI-Reflexion (Kristian Popov) . . . 12
3.5.1 KI als Co-Developer (Kristian Popov) . . . 12
3.5.2 Automatisierte Aufgabengenerierung (Kristian Popov) . . . 13
3.5.3 Kritische Reflexion (Kristian Popov) . . . 13

4 Designentscheidungen der Fachmodule **14**
4.1 Turingmaschine (Richard Schröder) . . . 14
4.1.1 Entscheidungen in der Architektur (Richard Schröder) . . . 14
4.1.2 Entscheidungen im UI (Richard Schröder) . . . 15
4.1.3 Andere Entscheidungen (Richard Schröder) . . . 15
4.2 Endliche Automaten (Kristian Popov) . . . 16
4.2.1 Entscheidungen in der Architektur (Kristian Popov) . . . 16
4.2.2 Entscheidungen im UI (Kristian Popov) . . . 17
4.2.3 Andere Entscheidungen (Kristian Popov) . . . 18
4.3 Reguläre Sprachen (Kristian Popov) . . . 18
4.3.1 Entscheidungen in der Architektur (Kristian Popov) . . . 19
4.3.2 Entscheidungen im UI (Kristian Popov) . . . 20
4.3.3 Andere Entscheidungen (Kristian Popov) . . . 20
4.4 Zahlensysteme (Enrico Ebert) . . . 21
4.4.1 Anforderungen und Herausforderungen (Enrico Ebert) . . . 21

4.4.2 Architektur (Enrico Ebert) . . . 21
4.4.3 UI-Entscheidungen (Enrico Ebert) . . . 22
4.4.4 Organisation und State Management (Enrico Ebert) . . . 22
4.5 Moore- und Mealy-Automaten (Markus Nöth) . . . 22
4.5.1 Entscheidungen in der Architektur (Markus Nöth) . . . 23
4.5.2 Entscheidungen im UI (Markus Nöth) . . . 23
4.5.3 Andere Entscheidungen (Markus Nöth) . . . 24
4.6 Disjunktive Normalform (Duy Tan Pham) . . . 24
4.6.1 Entscheidungen in der Architektur (Duy Tan Pham) . . . 24
4.6.2 Entscheidungen im UI (Duy Tan Pham) . . . 25
4.6.3 Andere Entscheidungen (Duy Tan Pham) . . . 26
4.7 Encoding (Enrico Ebert) . . . 26
4.7.1 Anforderungen und Herausforderungen (Enrico Ebert) . . . 26
4.7.2 Architektur (Enrico Ebert) . . . 26
4.7.3 Übungsmodus (Enrico Ebert) . . . 27
4.7.4 UI-Entscheidungen (Enrico Ebert) . . . 27
4.8 Java-Programmierumgebung (Markus Nöth) . . . 27
4.8.1 Entscheidungen in der Architektur (Markus Nöth) . . . 27
4.8.2 Entscheidungen im UI (Markus Nöth) . . . 28
4.8.3 Andere Entscheidungen (Markus Nöth) . . . 29

5 Evaluation & Projekterfahrung 29
5.1 Qualitätssicherung (Enrico Ebert) . . . 29
5.1.1 CI/CD-Pipeline (Enrico Ebert) . . . 29
5.1.2 Testing-Strategie (Enrico Ebert) . . . 29
5.1.3 Manuelle Prüfung (Enrico Ebert) . . . 30
5.1.4 Erfahrungen (Enrico Ebert) . . . 30
5.2 Abgleich mit dem Lastenheft (Kristian Popov) . . . 30
5.3 Lessons Learned (Alle) . . . 31
5.4 Design-Iterationen (Duy Tan Pham) . . . 33

6 Fazit & Ausblick 34
6.1 Zusammenfassung der Projektergebnisse (Alle) . . . 34
6.2 Perspektiven (Kristian Popov) . . . 36

Anhang i
Abkürzungsverzeichnis . . . i
Literatur . . . ii

Abbildungsverzeichnis

1 Architektur der Simulationsengine . . . 5
2 Erzeugung und Prüfung von Aufgabenvarianten . . . 13
3 Schichten des Simulators für endliche Automaten . . . 17
4 Kopplung von Übergangstabelle und Zustandsdiagramm . . . 18
5 Aufbau des Simulators für reguläre Sprachen . . . 19
6 Fachlicher Datenfluss im DNF-Bereich . . . 24

Tabellenverzeichnis

1 Einleitung

Die vorliegende Projektarbeit entstand im Sommersemester 2026 an der THWS unter Betreuung von Prof. Dr. Braun. Ziel des Projekts war die Konzeption und Umsetzung einer interaktiven Lernplattform für das Modul „Grundlagen der Informatik“.

1.1 Problemstellung (Enrico Ebert)

Die vermittelten Konzepte des Moduls „Grundlagen Informatik” stellen Studierende vor erhebliche Herausforderungen. Themen wie Turing-Maschinen, endliche Automaten, Moore- und Mealy-Automaten, Zahlensysteme sowie die Disjunktive Normalform sind durch einen hohen Abstraktionsgrad gekennzeichnet. Studierende müssen nicht nur die formalen Definitionen verstehen, sondern auch die dynamische Ausführung und Zustandsübergänge mental nachvollziehen können.

Erfahrungswerte zeigen, dass insbesondere die Visualisierung von Zustandsübergängen, das Verständnis der Bandbewegung bei Turing-Maschinen und die Umwandlung zwischen verschiedenen Automatenmodellen zu Verständnisschwierigkeiten führen. Traditionelle Lehrmethoden bieten oft nur statische Darstellungen in Vorlesungsfolien, die den dynamischen Charakter dieser Modelle nicht ausreichend vermitteln. Zudem fehlt es an interaktiven Übungsmöglichkeiten, die sofortiges Feedback geben und selbstständiges Experimentieren ermöglichen.

Die disjunktive Normalform erfordert das Verständnis komplexer logischer Umformungen, die ohne schrittweise Visualisierung schwer nachvollziehbar sind. Ähnlich verhält es sich mit Zahlensystemen und IEEE 754-Gleitkommazahlen, bei denen die binäre Repräsentation und die damit verbundenen Rundungsfehler für Anfänger schwer greifbar sind.

Aus diesen Erkenntnissen ergibt sich die Notwendigkeit einer interaktiven Lernplattform, die abstrakte Konzepte visualisiert, schrittweise Simulationen ermöglicht und Studierenden eigenständiges Üben mit direktem Feedback erlaubt.

1.2 Zielsetzung der Arbeit (Duy Tan Pham)

Ziel dieser Arbeit ist die Konzeption und Entwicklung einer webbasierten Lern- und Übungsplattform. Sie richtet sich vor allem an Studierende, die im Modul „Grundlagen Informatik“ mit abstrakten und fehleranfälligen Themen arbeiten. Die Plattform soll diese Inhalte verständlich aufbereiten und das selbstständige Lernen, Üben und Kontrollieren von Lösungswegen unterstützen.

Die Anwendung gibt deshalb nicht nur fertige Ergebnisse aus. Interaktive Simulationen, Visualisierungen und schrittweise Erklärungen machen sichtbar, wie ein Ergebnis entsteht. Beispiele zeigen vorbereitete Lösungswege. In den Übungen bearbeiten die Lernenden Aufgaben selbst und erhalten eine unmittelbare fachliche Rückmeldung. Freie Simulationen ermöglichen es zusätzlich, eigene Eingaben und Modelle auszuprobieren.

Die visuelle Darstellung soll die formale Schreibweise ergänzen. Tabellen eignen sich für eine genaue Eingabe und Prüfung. Graphen, Diagramme und hervorgehobene Zwischenschritte machen dagegen Zustände, Übergänge und andere Zusammenhänge leichter erkennbar. Je nach Fachthema werden deshalb unterschiedliche Darstellungen miteinander verbunden.

Die einzelnen Themen sollen nicht wie voneinander getrennte Werkzeuge wirken. Eine gemeinsame Navigation, ein einheitlicher Aufbau und wiederkehrende Bedienmuster verbinden sie zu einer Plattform. Die Anwendung läuft vollständig im Browser und benötigt weder eine Installation

noch ein Benutzerkonto oder eine serverseitige Verarbeitung. Die Oberfläche ist mehrsprachig. Eine helle und eine dunkle Darstellung unterstützt unterschiedliche Nutzungssituationen.

Kurze Tutorialvideos erleichtern den Einstieg in die einzelnen Themenbereiche. Sie zeigen die jeweilige Oberfläche und die wichtigsten Bedienabläufe, bevor die Lernenden mit den Beispielen und Übungen beginnen.

Der fachliche Schwerpunkt liegt auf den Themen aus Grundlagen Informatik. Sie bilden den Kern der Plattform und geben den Aufbau der einzelnen Themenbereiche vor. Im Projektverlauf kam mit Programmieren 1 ergänzend ein Lernbereich für Java hinzu, der denselben Aufbau nutzt.

1.3 Methodik (Markus Nöth)

Die Entwicklung der Plattform lief in wöchentlichen Zyklen ab. Zu Projektbeginn teilten wir die Anforderungen in fachliche Module und übergreifende Plattformdienste auf, für die einzelne Teammitglieder die Verantwortung übernahmen. Als organisatorischer Ansprechpartner kümmerte ich mich um den internen Austausch im Team, war bei den Terminen anwesend und klärte externe Fragen per E-Mail, beispielsweise zur Bereitstellung über GitLab Pages. Die Termine für die Abstimmungen gab meist Prof. Dr. Braun anhand seiner zeitlichen Verfügbarkeit vor.

Für die Versionsverwaltung und das Aufgabenmanagement nutzten wir das GitLab der THWS. Aufgaben und Fehlerberichte erfassten wir dort als Work Items mit konkreten Akzeptanzkriterien und wiesen sie Milestones zu. Die Programmierung erfolgte auf separaten Feature-Branches, die über Merge Requests in den `development`-Branch zusammengeführt wurden. Jeder Merge Request erforderte mindestens ein Review durch ein anderes Teammitglied sowie den fehlerfreien Durchlauf der CI-Pipeline mit Typprüfung, Unit-Tests und Linter.

In wöchentlichen Status-Meetings stimmte das Team Schnittstellen und gemeinsame Komponenten ab. In Reviews mit dem Auftraggeber Prof. Dr. Peter Braun führten wir lauffähige Zwischenstände vor. Das direkte Feedback floss in die folgenden Arbeitsschritte ein. So ließen sich Missverständnisse früh klären und didaktische Anforderungen, etwa die schrittweise Ausführung und die visuelle Darstellung der Rechenschritte, direkt in den Simulatoren umsetzen.

2 Übergreifende Architektur & Systemdesign

2.1 Technologiestack (Richard Schröder)

Die Anwendung ist als Client-Webanwendung ohne eigenes Anwendungsbackend umgesetzt. Simulationen, Java-Kompilierung und Speicherung laufen im Browser. Ein Webserver liefert nur die statischen Build-Artefakte aus; die Anwendung kann deshalb auch lokal bereitgestellt werden. React und TypeScript wurden zu Beginn des Projekts festgelegt, React 19 bildet die Benutzeroberfläche, während TypeScript unter anderem Simulator-Konfigurationen, Laufzeitzustände und Transfer-Codecs typisiert. Der React Router lädt die Komponenten der Seiten und Themen erst beim Aufruf der jeweiligen Route, sodass sie nicht Teil des initial geladenen JavaScript-Bundles sein müssen.

Vite dient als lokaler Entwicklungsserver und erzeugt den statischen Build für das Deployment. Die Unit-Tests werden mit Vitest ausgeführt.

Das Styling basiert auf Tailwind CSS 4, Utility-Klassen und zentralen CSS-Variablen für Farben und weitere Darstellungswerte. Das Projekt enthält die Konfiguration und die Tailwind-Styles

von `shadcn/ui`, verwendet aber auch eigene Komponenten unter den Shared Components. Lucide React stellt die Icons als React-Komponenten bereit.

Zustand verwaltet den eingeklappten Zustand der Sidebar, Theme, Schriftart und den aktiven Simulatortransfer. Theme und Schriftart werden im Local Storage gespeichert; der Sidebar-Zustand wird nicht gespeichert. Die fachlichen Zustände der Simulatoren verbleiben in den jeweiligen Hooks und Domänenmodellen. Der globale Zustand bleibt damit auf Werte beschränkt, die mehrere Komponenten benötigen.

Für die Internationalisierung kommen i18next und react-i18next zum Einsatz. Die Anwendung unterstützt Deutsch, Englisch, Türkisch und Ukrainisch.

Im Java-Bereich ist Monaco als Editor eingebunden. Ein OpenJDK-Compiler kompiliert den Java-Code mit `javac` in einem Web Worker. TeaVM überführt den kompilierten Code anschließend in WebAssembly, das der Browser ausführt.

Fallow prüft den TypeScript-Code auf Dead Code, Duplikate und Code-Health einschließlich Komplexität und großer Funktionen sowie auf Architektur- und Abhängigkeitsprobleme. Die Ergebnisse weisen auf nicht verwendete Bestandteile, Kopien und auffällige Abhängigkeiten hin.

Die Versionsverwaltung erfolgt mit Git. Das zentrale Repository und die automatisierten Abläufe liegen im GitLab der THWS. GitLab CI führt Formatierungs- und Lint-Prüfungen, Unit-Tests, den Build, E2E-Tests mit Playwright sowie die Lighthouse-Prüfung aus. Die Umsetzung der Pipeline und des GitLab Runners wird in Kapitel 5.1 behandelt.

2.2 Modulare Softwarearchitektur (Markus Nöth)

Die Architektur der Plattform ist modular aufgebaut, damit sich neue Lernmodule und Simulatoren ohne Änderungen an bestehenden Kernkomponenten einfügen lassen. Sie trennt die globale Rahmenstruktur, die deklarative Modulkonfiguration und die fachspezifischen Implementierungen voneinander.

Die Komponente `MainLayout` bildet den äußeren Rahmen der Anwendung. Sie bindet `Header`, `Sidebar` und `Footer` ein und rendert die aktive Route über ein React-Router-`Outlet`. Ein umschließendes `Suspense`-Element mit der Komponente `LoadingFallback` fängt Ladezeiten beim asynchronen Nachladen von Seiten ab. Den Zustand der Seitenleiste (ein- oder ausgeklappt) verwaltet ein zentraler Zustand-Store (`useUiStore`). Dadurch führen Änderungen an der Navigation nicht zum erneuten Rendern des Hauptinhalts.

Die Konfiguration der Lerninhalte teilt sich in zwei Dateien auf, um zirkuläre Abhängigkeiten zu vermeiden und Code-Splitting zu unterstützen:

- **Metadaten-Katalog (`moduleCatalogData.ts`):** Enthält reine Datenstrukturen ohne React-Abhängigkeiten. Hier sind die Moduldefinitionen (`LearningModuleDefinition`) und Themenbereiche (`ModuleTopicDefinition`) mit Pfaden, Lucide-Icons, Farbwerten, Opencast-Video-URLs und getypten Lokalisierungsschlüsseln (`TranslationKey`) hinterlegt. Jedes Thema erhält hier zusätzlich seine numerische Speicher-ID aus `topicStorageIds.ts`.
- **Komponenten-Binding (`moduleCatalog.tsx`):** Bindet Seitendateien asynchron per `React.lazy()` ein. Es weist die Komponenten in den Maps `modulePageElements` sowie `topicPageElements` zu und übergibt das Routen-Array `learningRoutes` an den Router.

Für die Ansicht eines Themenbereichs stellt die Shared-Komponente `ModuleTopicPage` eine einheitliche Struktur bereit. Sie steuert Kopfbereich, Tab-Navigation, Übungsauswahl und Vollbildmodus. Die Inhalte sind auf vier Reiter verteilt: Übersicht (`OverviewTab`), Beispiele

(`ExamplesTab`), Übungen (`ExercisesTab`) und Simulation (`SimulationTab`). Der Simulations-Reiter nutzt den Rahmen `SimulationWorkspace`, der Steuerelemente, Informationsbereiche und die jeweilige Simulator-Komponente einbindet.

Um ein neues Lernmodul mit einem Thema hinzuzufügen, sind folgende Schritte erforderlich:

1. **Verzeichnisstruktur anlegen:** Unter `app/src/pages/<ModuleName>/<TopicName>/` werden die vier Reiter-Komponenten und die Datei `<TopicName>Page.tsx` erstellt, welche die `ModuleTopicPage` aufruft.
2. **Speicher-ID vergeben:** In `topicStorageIds.ts` wird eine eindeutige Zahl im Objekt `TOPIC_STORAGE_IDS` hinterlegt. Diese ID trennt die Speicherstände der Themen im Local Storage und verhindert Konflikte beim automatischen Sichern oder Zurücksetzen.
3. **Katalogeintrag definieren:** In `moduleCatalogData.ts` wird das Thema mit Pfad, Icon und Metadaten dem gewünschten Modul zugeordnet.
4. **Lazy-Route einbinden:** In `moduleCatalog.tsx` wird die Seite mit `React.lazy()` importiert und in `topicPageElements` eingetragen.
5. **Lokalisierungsdateien anlegen:** Unter `app/src/i18n/locales/` wird jeweils eine themenspezifische Übersetzungsdatei für Deutsch und Englisch (z. B. `de/<topic>.json`) mit allen Bezeichnungen und Aufgaben angelegt.
6. **Sprachdateien registrieren:** In `app/src/i18n/index.ts` werden die beiden JSON-Dateien importiert und in den Objekten `de` und `en` als neuer Namespace der i18n hinterlegt.

Durch diesen Ablauf erzeugt das System Routen, Navigationseinträge und Breadcrumbs automatisch aus den Metadaten. Der Browser lädt den Quellcode eines Themas erst herunter, wenn der Benutzer die Seite im Browser aufruft.

2.3 Unified UI-Konzept (Duy Tan Pham)

Das Unified UI-Konzept verbindet die unterschiedlichen Themenbereiche zu einer einheitlichen Plattform, statt sie wie einzelne Systeme nebeneinanderzustellen. Kopfbereich, Seitenleiste und Fußbereich bilden einen gemeinsamen Rahmen. Die Navigation wird aus dem zentralen Modulkatalog erzeugt. Dadurch führen dieselben Informationen zu den Einträgen in der Seitenleiste, den Routen und den Übersichtsseiten.

Die Themenbereiche verwenden mit `ModuleTopicPage` denselben grundlegenden Seitenaufbau. Die Inhalte sind in Übersicht, Übungen, Beispiele und Simulation gegliedert. Gemeinsame Komponenten stellen unter anderem Aufgaben, Tabellen, Dialoge, Rückmeldungen und Simulationsbereiche dar. Ein ausgewähltes Beispiel oder eine Aufgabe kann dadurch nach einem wiederkehrenden Ablauf in der Simulation geöffnet und bearbeitet werden.

Auch die visuelle Gestaltung wird zentral zusammengehalten. CSS-Variablen legen Farben und Zustände für die helle und dunkle Darstellung fest. Gemeinsame Stilregeln sorgen für einheitliche Abstände und Typografie. Themenspezifische Akzentfarben erleichtern die Orientierung, ohne den gemeinsamen Stil zu verlassen. Wiederkehrende Symbole, Schaltflächen und Rückmeldungen verhalten sich in den verschiedenen Bereichen gleich. Eingabe, Ergebnis und Erklärung bleiben dabei erkennbar voneinander getrennt.

Die gemeinsame Oberfläche schreibt den Fachthemen nicht dieselbe Visualisierung vor. Tabellen bleiben dort erhalten, wo Zustände, Werte oder Übergänge genau eingegeben und geprüft werden müssen. Graphen, Logikdiagramme und schrittweise Hervorhebungen ergänzen diese Darstellung,

wenn Beziehungen oder Abläufe dadurch leichter zu erkennen sind. So kann jedes Fachthema die passende Darstellungsform nutzen und trotzdem den gemeinsamen Bedienmustern folgen.

Die gemeinsame Internationalisierung stellt die Oberfläche in mehreren Sprachen bereit. Sie schafft zugleich eine einheitliche Grundlage für die verwendeten Fachbegriffe. Die Oberfläche ist zunächst für die Arbeit am Desktop ausgelegt und passt sich kleineren Bildschirmbreiten an. Komplexe Tabellen und Simulatoren behalten dabei den Platz, den sie für eine verständliche Darstellung benötigen.

Für Tutorialvideos gibt es einen gemeinsamen Bereich und eine wiederverwendbare Videokomponente. Dadurch sind die Einführungsvideos in derselben Form in die Themenübersichten eingebunden.

Das gemeinsame Bedienkonzept vereinheitlicht nur die wiederkehrenden Teile. Fachliche Inhalte, Prüfregeln und spezielle Visualisierungen bleiben im jeweiligen Themenbereich. So entsteht eine erkennbare Plattform, ohne die unterschiedlichen Anforderungen der Fachthemen in eine zu allgemeine Komponente zu zwingen.

2.4 Simulator-Architektur (Richard Schröder)

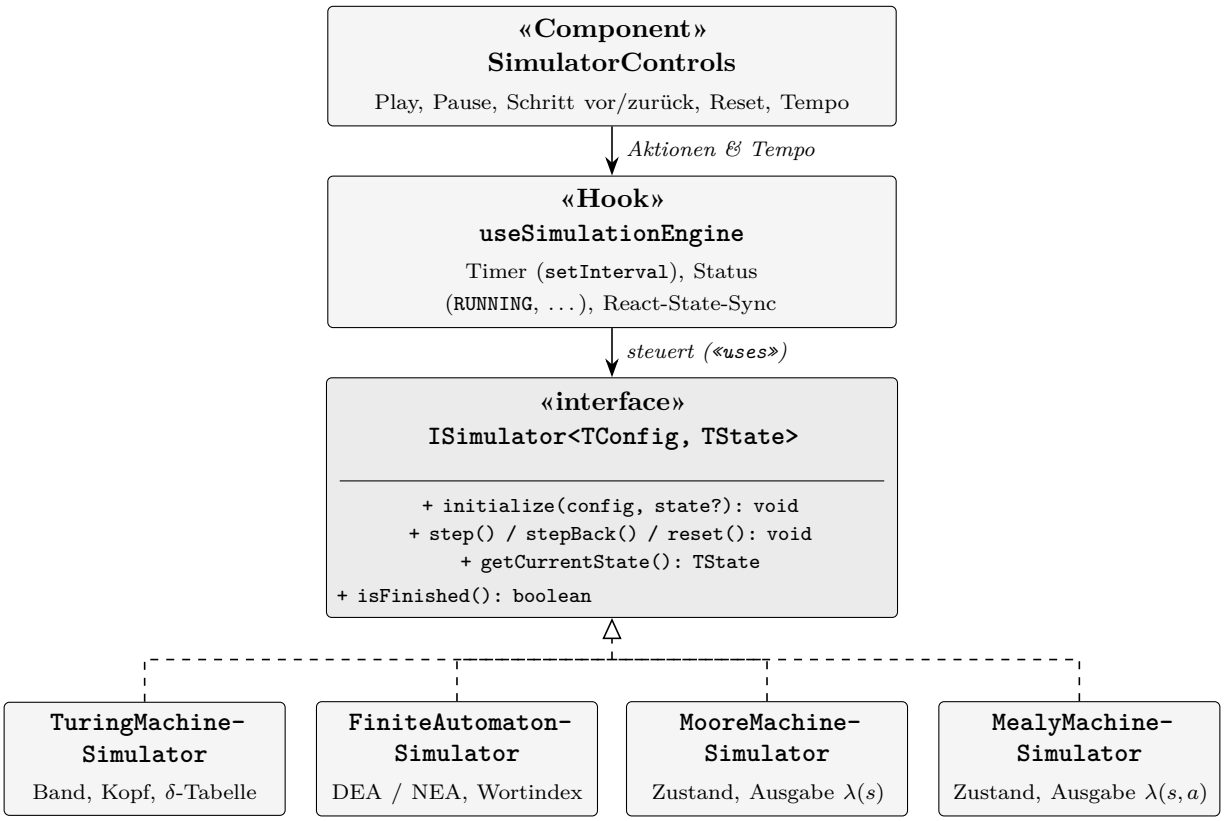


Abbildung 1: Architektur der Simulationsengine. Die UI-Komponente `SimulatorControls` interagiert mit dem Hook `useSimulationEngine`. Dieser steuert über das gemeinsame Interface `ISimulator` die Ausführung und synchronisiert den Laufzeitzustand, während die modulspezifischen Simulator-Klassen die fachlichen Zustandsübergänge implementieren.

Turingmaschinen, endliche Automaten sowie Moore- und Mealy-Automaten verwenden dieselbe Simulationsengine. Ihre Klassen implementieren `ISimulator<TConfig, TState>`. Das Interface umfasst Initialisierung, Schritt, Rückschritt, Reset, Konfiguration, aktuellen Zustand und eine

Prüfung ob der Haltezustand erreicht wurde. Es verbindet damit die fachliche Simulationslogik mit der Steuerung, ohne eine konkrete Darstellung vorzuschreiben.

Die Gesamtausführung folgt dabei dem in Abbildung 1 dargestellten Schichtenmodell: Der Hook `useSimulationEngine` verwaltet Ausführungsstatus, Geschwindigkeit, Browser-Timer und Fehlerbehandlung. Nach einem Schritt übernimmt er den Zustand des Simulators in den React-State. Alle vier Maschinentypen erhalten über den Hook und die gemeinsamen Simulator-Controls Play, Pause, Schritt, Rückschritt, Reset und Geschwindigkeitssteuerung, während ihre Klassen die jeweiligen Zustandsübergänge ausführen.

Ein weiterer Maschinentyp benötigt eine Implementierung des Interfaces, einen modulspezifischen Hook-Aufruf und die Anbindung an eine Laufzeitansicht. Die allgemeine Timerlogik und die vorhandenen Simulator-Controls können dabei unverändert bleiben.

2.5 Datenmodellierung und Transfer (Richard Schröder)

Die Simulatoren trennen die editierbare Konfiguration von den Daten einer laufenden Ausführung. `TMProgram`, `FAProgram`, `MooreProgram` und `MealyProgram` enthalten Zustände, Alphabet, Übergänge und Start- bzw Endzustand. Die Übergänge liegen als verschachtelte `Record`-Strukturen vor, Zustandsnamen und Symbole werden als Zeichenketten gespeichert. Bei Turingmaschinen sowie Moore- und Mealy-Automaten gehört das Eingabewort zum Programm, bei endlichen Automaten zum Laufzeitzustand.

`TMRuntime` speichert aktuellen Zustand, Band, Kopfposition und Haltegrund. `FARuntime` hält die Position im Eingabewort und den Zustandsverlauf. `MooreRuntime` und `MealyRuntime` speichern Zustands- beziehungsweise Übergangsverläufe; bei Mealy-Automaten kommen die ausgegebenen Symbole hinzu. Die Schrittausführung verändert diese fachlichen Werte. Geöffnete Dialoge und Animationen bleiben dagegen als kurzfristige Zustände in den React-Komponenten.

Das DNF-Modul führt Ausdrücke, Wahrheitstabellen und Antworten getrennt. Im Zahlensystem-Simulator bündelt `NumberSystemState` mit `activeMode` die Daten für Ganzzahlen, negative Zahlen, Brüche, IEEE 754 und arithmetische Aufgaben. Der Java-Simulator überträgt den Quelltext als Zeichenkette.

Für den Austausch definiert `ISimulatorTransferCodec<TValue>` die Methoden `exportValue` und `importValue`. Sie erzeugen einen Transfer-String beziehungsweise rekonstruieren daraus den fachlichen Wert. Beide Methoden dürfen direkt oder als `Promise` antworten, weil Formate wie der Java-Codec die asynchronen Browser-APIs `CompressionStream` und `DecompressionStream` verwenden. Local Storage selbst arbeitet synchron.

`SimulatorStringTransferBox` stellt das Eingabefeld und die Aktionen zum Erzeugen, Kopieren und Laden eines Transfer-Strings bereit. Das Training Center speichert den vom fachlichen Codec erzeugten Payload getrennt mit einem Bezeichner im Local Storage. Der Autosave exportiert den aktuellen Wert standardmäßig alle zehn Sekunden und beim Aufräumen des Hooks. Beim Import wird der Bezeichner geprüft, bevor der Wert an den fachlichen Codec gelangt.

Mehrere Codecs schreiben ihre Daten mit `BitWriter` und lesen sie mit `BitReader`. Die Bits werden als Base64URL-Zeichenkette übertragen. Versionsfelder, Größenlimits, IDs und Füllbits begrenzen und prüfen das Format. Gegenüber einem lesbaren Textformat entstehen kürzere Transfer-Strings; ohne passenden Decoder ist der Inhalt jedoch nicht lesbar.

Der Turingmaschinen-Codec z. B. normalisiert Programme, entfernt nicht verwendete Zustände und Symbole und vergibt numerische IDs. Für Symbole wählt er zwischen einer festen Registry und UTF-8. Für Übergänge erzeugt er dichte und dünn besetzte Kandidaten, jeweils mit und

ohne Dictionary, und exportiert den Kandidaten mit der geringsten Bitlänge. So richtet sich die Länge des Transfer-Strings nach der Struktur des jeweiligen Programms.

Der Java-Codec verwendet direkte Tokens, deflatierte Tokens oder deflatiertes UTF-8 und wählt die kürzeste erzeugte Darstellung. Beim Import prüft er Größenlimits sowie das Token- und UTF-8-Format.

3 Einsatz von Künstlicher Intelligenz im Projekt

3.1 KI-Reflexion (Markus Nöth)

3.1.1 KI als Co-Developer (Markus Nöth)

Während der Entwicklung kamen ChatGPT, Codex in WebStorm sowie Claude für Konzeption, Entwurf und Refactorings zum Einsatz. Für die direkte Unterstützung in der IDE nutzte ich AntiGravity, das in WebStorm eingebunden war. Um die Zusammenhänge und Abhängigkeiten in der Codebasis schnell zu überblicken, setzte ich das Wissensgraphen-Werkzeug Graphify [1] ein. Es half dabei, zentrale Knotenpunkte (God Nodes) im Modul- und Routensystem zu identifizieren und Kopplungen zwischen Komponenten vor größeren Umbauten sichtbar zu machen. Ergänzend dienten Stitch und Figma für UI-Entwürfe sowie eine Playwright-MCP-Anbindung für automatisierte Browsertests.

Um unpassende Vorschläge zu vermeiden, teilte ich die Arbeit mit generativen Werkzeugen in fünf feste Schritte ein:

1. **Konzeption:** Genaue Beschreibung von Zielzustand, Datenstrukturen und Schnittstellen vor der ersten Anfrage.
2. **Planungsphase:** Erstellung eines strukturierten Umsetzungsplans im Planungsmodus des Werkzeugs und anschließende manuelle Prüfung der vorgeschlagenen Schritte.
3. **Prompt-Entwicklung:** Formulierung des eigentlichen Entwicklungsprompts, der gezielt Abschnitte aus dem Plan aufgreift und geforderte Anpassungen präzisiert.
4. **Inkrementelle Umsetzung:** Schrittweise Implementierung in kleinen, überschaubaren Code-Einheiten.
5. **Verifikation:** Prüfung des erzeugten Codes über TypeScript-Kompilierung, Unit-Tests, Git-Diffs, E2E-Tests und Linting.

Beim Entwurf der Benutzeroberflächen, beispielsweise für die Zustandstabellen der Automaten, nutzte ich Git für den direkten Variantenvergleich: Ich ließ alternative Layoutansätze als getrennte Entwürfe erstellen und wechselte mit `git stash` im laufenden Vite-Entwicklungsserver zwischen den Varianten hin und her. Der unmittelbare Vergleich im Browser zeigte Bedienprobleme schnell auf, bevor eine Variante fest übernommen wurde.

Für die Prüfung interaktiver Abläufe nutzte ich Playwright über das Model Context Protocol (MCP). Das Modell steuerte das kompilierte Frontend im Browser, klickte Pfade in den Simulatoren ab, löste Zustandswechsel aus und untersuchte Fehlermeldungen im Monaco-Editor. Auf diese Weise ließen sich Timing-Probleme und unerwartete UI-Zustände aufdecken, die bei reiner Code-Inspektion leicht übersehen werden.

3.1.2 Aufgabengenerierung und manuelle Prüfung (Markus Nöth)

Für die Moore-Automaten und die Java-Programmierumgebung wurden Übungsaufgaben mithilfe von Sprachmodellen entworfen. Bei den Automaten dienten formale Ablaufbeschreibungen als Ausgangspunkt, etwa für Drehkreuze oder Ampelsteuerungen. Frühe Prompts erzeugten dabei unvollständige Testmatrizen oder ungültige Übergänge. Abhilfe schufen ein festes JSON-Schema mit Validierungsskripten und Vorgabedateien.

Für das Java-Modul wurden Aufgabenstellungen zu Verzweigungen, Schleifen und Arrays entworfen. Um didaktische Qualität und fachliche Korrektheit sicherzustellen, verließ ich mich nicht allein auf Skripte: Sämtliche generierten Aufgaben für Moore-Automaten und Java wurden von mir per Hand im Simulator durchgespielt, Schritt für Schritt getestet und gegen die Musterlösungen abgeglichen.

3.1.3 Dokumentation und Markdown-Dateien (Markus Nöth)

Die Aufgabenstellung von Prof. Dr. Braun sah vor, generative KI intensiv im Projekt einzusetzen. Neben der Programmierung nutzte ich Sprachmodelle deshalb auch beim Schreiben der Dokumentation und der Projektnotizen. Ich gab stichpunktartige Notizen in Prompts ein, um erste Textentwürfe zu formulieren, eigene Sätze sprachlich zu glätten oder Markdown-Dateien wie `handover.md` vorzustrukturieren. Die erstellten Textabschnitt habe ich selbst gegengelesen, an den tatsächlichen Stand angepasst und falls nötig korrigiert, da ich nur so die Verantwortung für die Inhalte meiner Teile des Projekts übernehmen kann.

3.1.4 Kritische Reflexion (Markus Nöth)

Sprachmodelle sparten Zeit bei Standardstrukturen, Testfällen und UI-Gerüsten. Gleichzeitig zeigten sich zwei wesentliche Grenzen:

- **Hang zur Überkomplexität:** Werkzeuge schlugen mehrfach überdimensionierte Hilfsklassen oder globale Zustände vor. Ein generierter Entwurf für ein dynamisches, zustandsbehaftetes Routingsystem wurde verworfen und durch die zwei einfachen Dateien `moduleCatalogData.ts` und `moduleCatalog.tsx` ersetzt.
- **Grenzen bei Laufzeitumgebungen und WebAssembly:** Für die browserbasierte Java-Ausführung schlug die KI anfangs Java-Transpiler oder oberflächliche Mocks vor. Nach eigener Recherche wählte ich TeaVM aus, um Bytecode im Browser über Wasm auszuführen. Bei der Umsetzung zeigte sich jedoch, dass blockierende Eingaben über `Scanner` und `System.in` in der Wasm-Sandbox des Browsers nicht synchron angehalten werden können. Da die Sprachmodelle hierfür keine funktionierende Lösung lieferten, entwickelte ich die Quelltext-Vorverarbeitung mit `ScannerInputCursor`, die Aufrufe vor dem Kompilieren deterministisch durch Werte aus dem Eingabefeld ersetzt.

Die Werkzeuge waren eine nützliche Hilfe im Entwicklungsprozess, erforderten jedoch klare architektonische Vorgaben und eine ständige manuelle Kontrolle der Ergebnisse.

3.2 KI-Reflexion (Richard Schröder)

3.2.1 KI als Co-Developer (Richard Schröder)

Ich setzte Codex und Antigravite in WebStorm und Zed sowie lokal über Ollama ausgeführte Modelle für Planung, Implementierung und Code-Reviews ein. Gemini nutzte ich ergänzend für Recherchen und Formulierungsvorschläge. Die Ergebnisse dienten jeweils als Entwurf. Ar-

chitekturänderungen, bestehende Abhängigkeiten und Tests prüfte ich vor der Übernahme im Projekt.

Mein meistens verwendeter Ablauf bestand aus drei Phasen. In der Planung beschrieb ich den Zielzustand, prüfte den erzeugten Architekturvorschlag und zerlegte ihn in Arbeitspakete. In der Umsetzung bearbeitete ich diese Pakete einzeln und verglich jede Änderung mit dem geplanten Aufbau. Für die unabhängige Prüfung verwendete ich einen neuen Chat und kontrollierte die Befunde anschließend selbst im Code und in den Unit- und E2E-Tests.

Bei einem getrennten Code-Review der Turingmaschinenimplementierung fand Codex zwei Fehler, die von den vorhandenen Tests nicht abgedeckt waren. Nach dem Austausch des Anfangsbandes wurde das Eingabealphabet nicht neu abgeleitet. Außerdem akzeptierte die Lösungskontrolle eine passende Ausgabe auch dann, wenn die Maschine wegen eines fehlenden Übergangs statt in einem vorgesehenen Haltezustand stoppte. Ich prüfte beide Befunde, korrigierte die Logik und ergänzte Regressionstests für die beiden Fälle. Der Review fand zudem nicht verwendeten Produktivcode, Code Dopplungen und fehlende direkte Tests der Bandansicht.

3.2.2 Automatisierte Aufgabengenerierung (Richard Schröder)

Bei den ersten erzeugten Turingmaschinenaufgaben entstanden sowohl ungültige JSON-Ausgaben als auch Maschinen, die sich nicht wie vorgesehen ausführen ließen. Ich gab den Modellen danach gültige Aufgabenobjekte als Beispiele und prüfte jede erzeugte Maschine durch Import und Simulation. Die Aufgaben werden als geprüfte Szenarien in der Anwendung gespeichert.

Für die Übertragung von Aufgaben aus Skript und Übungsblättern verwendete ich bestehende JSON-Beispiele zusammen mit einem Screenshot der jeweiligen Lösung. Diese Eingaben halfen den Modellen, Struktur und Inhalt der Vorlage zuzuordnen. Vor der Aufnahme kontrollierte ich weiterhin das JSON-Format, die Übergänge und die erwarteten Ergebnisse der Validierungsfälle.

3.2.3 Kritische Reflexion (Richard Schröder)

Für mich stellte der Einsatz von KI im Projekt eine erhebliche Unterstützung dar, insbesondere um Entwicklungsaufgaben schneller zu strukturieren, Implementierungen vorzubereiten und alternative Lösungsansätze zu prüfen. Dadurch konnte ich effizienter arbeiten, ohne jeden Entwurf oder jede technische Detailentscheidung vollständig von Grund auf manuell ausarbeiten zu müssen. Den größten Nutzen hatte ich, wenn ich die Anfragen klar eingrenzte und die Ergebnisse anschließend überprüfte.

Eine zentrale Einschränkung bestand für mich darin, dass ich zu große Aufgaben nicht zuverlässig in einem einzigen Schritt bearbeiten konnte. Wenn ich beispielsweise versuchte, den ersten Entwurf der Turingmaschine mit allen Details auf einmal generieren zu lassen, gingen Teile der Anforderungen aus dem Kontext verloren oder wurden fehlerhaft umgesetzt. Daher zerlegte ich komplexe Aufgaben in kleinere Einheiten und bearbeitete sie nacheinander.

Darüber hinaus beobachtete ich gelegentlich, dass KI-Systeme einfache Problemstellungen unnötig komplex lösten. In solchen Fällen erhielt ich Vorschläge für umfangreiche Strukturen, zusätzliche Abstraktionen oder technische Umwege, obwohl eine deutlich einfachere Lösung ausgereicht hätte. Deshalb musste ich die vorgeschlagene Architektur und Implementierung immer kontrollieren.

Problematisch war für mich außerdem, dass KI-generierte Änderungen nicht immer an der fachlich passenden Stelle vorgenommen wurden. Ich stellte fest, dass allgemeine oder gemeinsam genutzte Komponenten angepasst wurden, obwohl eine lokale Änderung innerhalb des jeweiligen Moduls angemessener gewesen wäre. Solche Änderungen konnten andere Teile des Systems

beeinflussen. Mir wurde dadurch klar, dass ich KI-Unterstützung nur mit einem guten Verständnis der bestehenden Codebasis einsetzen kann.

3.3 KI Reflexion (Enrico Ebert)

Für die Entwicklung der Lernplattform wurden KI-Tools aktiv in den Softwareentwicklungsprozess integriert. Sie dienten als Unterstützung bei der Codegenerierung sowie der Konzeption der Module. Dieses Kapitel fasst die praktischen Erfahrungen zusammen und bewertet die Stärken und Schwächen generativer Modelle im Programmieralltag.

3.3.1 KI als Co-Developer (Enrico Ebert)

Als „Co-Developer“ kamen drei Werkzeuge zum Einsatz, die unterschiedliche Stärken zeigten:

- **`google/Gemma agentMode` (WebStorm):** Effektiv für lokales Refactoring und isolierte Features. Das modulübergreifende i18n-System wurde aufgrund fehlenden globalen Projektkontexts jedoch oft ignoriert.
- **`google/Antigravity Gemini 3.1 Pro`:** Hilfreich beim Prototyping und bei Entwicklungsplänen. Das Modell übersah jedoch oft technische Edge Cases. Die Einhaltung der Projektstruktur und i18n-Vorgaben musste iterativ erzwungen werden.
- **`OpenCode` (BigPickle):** Effizient für punktuelle Code-Anpassungen in Kombination mit dem integrierten Planungsmodus.

Gesamteindruck: Planung und Prototyping funktionierten gut, Kontextverständnis und Vollständigkeit fehlten oft. Daraus ergab sich folgender Workflow:

1. KI-Planungsmodus für den Umsetzungsplan nutzen.
2. Grundlegendes Code-Gerüst generieren lassen.
3. Code manuell anpassen (Typisierung, i18n und Architekturvorgaben sicherstellen).

3.3.2 Kritische Reflexion (Enrico Ebert)

Die Grenzen der KI zeigten sich vor allem bei exakter, deterministischer Logik. Bei komplexen mathematischen und bitweisen Operationen, wie Umrechnungen mit `BigInt`, IEEE-754 Bitmanipulationen oder dem Parsen von UTF-8-Sequenzen, lieferten die Modelle oft fehlerhaften Code, der manuell refaktorisiert werden musste.

Ein Hauptproblem war das Ignorieren von Edge Cases. Kritische Zustände wie NaN, $\pm$Infinity, Overlong Encodings bei UTF-8 oder genaue Überlauferkennung wurden meist übersehen und mussten durch eigene Unit- und E2E-Tests (Vitest, Playwright) abgesichert werden.

Bei größeren Komponenten stießen die Modelle zudem an die Grenzen ihres Kontextfensters. In komplexeren Dateien vergaß die KI regelmäßig existierende i18n-Keys, Shared Components und allgemeine Projektkonventionen, was ständige manuelle Korrekturen erforderte.

Fazit: KI-Tools beschleunigen Prototyping, Konzeption und das Schreiben von Boilerplate-Code enorm. Sie ersetzen aber weder architektonisches Verständnis noch die manuelle Qualitätssicherung. Der größte Nutzen lag in der strukturierten Planung und dem Refactoring kleiner, isolierter Code-Einheiten.

3.4 KI Reflexion (Duy Tan Pham)

Für meinen Teil, die DNF, habe ich durchgehend mit KI-Werkzeugen gearbeitet. Codex war das Hauptwerkzeug beim Entwickeln und direkt in VS Code eingebunden. Claude habe ich für Analyse, Refactorings sowie für Kontext- und Wissensdokumente genutzt. Die KI diente dabei nicht nur als Entwicklungswerkzeug, sondern auch zur Aufarbeitung der fachlichen Grundlagen. Das Verfahren von Quine-McCluskey habe ich mir zum Beispiel mit der KI erarbeitet, bevor ich es umgesetzt habe.

3.4.1 KI als Co-Developer (Duy Tan Pham)

Meine Arbeitsweise mit der KI war vor allem dialogisch und nicht befehlend. Statt fertige Vorgaben zu machen, habe ich die KI meist zuerst nach einer Einschätzung und einem Lösungsansatz gefragt, bevor etwas umgesetzt wurde. Die eigentliche Entscheidung ist dann schrittweise im Austausch entstanden. Dieses Nachfragen war zugleich meine Kontrolle. Weil ich die Vorschläge hinterfragt habe, bevor ich sie übernahm, blieb die fachliche Entscheidung bei mir, und Fehler fielen früher auf.

Neben Lösungsvorschlägen habe ich die KI gezielt nach Kritik am eigenen Code gefragt, etwa nach übersehenen Sonderfällen oder unsauberen Stellen. Eine feste Vorgabe war dabei die Einheitlichkeit. Der DNF-Bereich sollte den gemeinsamen Gestaltungs- und Interaktionsmustern der Lernplattform folgen.

Zusätzlich gab es feste Regeln, die die Arbeit mit der KI absicherten. Vorgegebene Skills und Anweisungen legten fest, wie die KI eine Aufgabe angehen und wie genau sie das Ergebnis danach prüfen sollte, abhängig davon, wie riskant eine Änderung war. Bei der Arbeit mit Claude blockierten außerdem zwei automatische Prüfungen gefährliche Befehle und das versehentliche Preisgeben von Zugangsdaten, bevor sie ausgeführt wurden.

3.4.2 Automatisierte Aufgabengenerierung (Duy Tan Pham)

Ein weiteres Einsatzfeld der KI war das Erstellen von Übungsaufgaben. Den Ausgangspunkt bildeten die Aufgaben aus dem Übungsblatt der Vorlesung. Für beide Aufgabenmodi und alle Schwierigkeitsstufen reichten diese Vorlagen jedoch nicht aus, sodass weitere Aufgaben nötig waren.

Dafür habe ich der KI die bereits vorhandenen Aufgaben als Kontext gegeben. Auf dieser Grundlage ließen sich weitere Aufgaben desselben Typs ergänzen, etwa mit anderen Variablen, einer anderen Wahrheitstabelle oder einem höheren Schwierigkeitsgrad. Die Aufgaben sind anschließend als feste Definitionen im Code hinterlegt und werden nicht erst während der Nutzung erzeugt.

Ein wichtiger Punkt war dabei, dass die KI keine Lösung mitliefern musste. Die erwarteten Ergebnisse berechnet die DNF-Fachlogik direkt aus den Aufgabendaten. Dadurch kann kein separat gepflegter Lösungsschlüssel von der Aufgabe abweichen. Syntax, Schwierigkeitsgrad und didaktische Eignung mussten trotzdem geprüft werden. Zu beurteilen blieb außerdem, ob sich eine ergänzte Aufgabe ausreichend von den bereits vorhandenen unterschied.

3.4.3 Kritische Reflexion (Duy Tan Pham)

Die wichtigste Erkenntnis war für mich, dass man sich bei deterministischer Informatik-Logik nicht blind auf die KI verlassen kann. Ein Beispiel ist die Minimierung. Die damalige Lösung hat die Restüberdeckung greedy gewählt. Erst meine konkrete Nachfrage, ob damit die literalminimale DNF übersehen werden kann, führte zur Diagnose und anschließend zur exakten Lösung. Ich

musste das Korrektheitskriterium also selbst benennen und einfordern. Von allein hat die KI es nicht sichergestellt.

Ein zweites Thema war das begrenzte Kontextfenster. In langen Chats ging Kontext verloren, deshalb habe ich mir Zusammenfassungen für neue Chats erstellen lassen und wichtige Fakten in eigene Wissensdokumente ausgelagert. Um das Repository herum entstand dadurch ein lokaler Arbeitsbereich mit Vorlesungsunterlagen, Übungsblättern, Lastenheft, Work-Item-Exporten, Videoquellen und ausgewählten Chatprotokollen. Kurze Handoff-Dateien und aufgabenspezifische Skills steuerten, welche Quellen ein neuer Agent für eine Aufgabe lesen und welche Prüfungen er durchführen sollte. Dadurch musste nicht jeder Chat den gesamten Projektverlauf neu einlesen. Die zusätzlichen Dateien waren dabei nur Orientierung und Prozessbelege. Technische Aussagen wurden weiterhin am aktuellen Code, an Tests oder an Commits geprüft. Mit wachsendem und älterem Kontext nahmen außerdem falsche Annahmen der KI zu. Ein neuer Chat mit einer kurzen, aktuellen Zusammenfassung lieferte deshalb oft zuverlässigere Antworten als ein sehr langer Verlauf.

Außerdem musste ich auf den Umfang der Anfragen achten. Zu große Aufgaben auf einmal wurden unzuverlässig, kleinere Schritte lieferten bessere Ergebnisse. Auch eine Tendenz zu unnötig komplexen Lösungen war ein wiederkehrendes Muster. Solche Vorschläge habe ich bewusst wieder zurückgenommen, etwa einen unnötigen Refactor der Eingabe-Komponenten.

Ein wiederkehrendes Risiko waren unbelegte technische Annahmen. Angaben zu externen Quellen oder zu Eigenschaften einzelner Dateien hat die KI sehr überzeugend formuliert, obwohl sie nicht zutrafen. Aufgefallen ist das erst, als ich diese Angaben mit passenden Werkzeugen an der Quelle selbst nachgeprüft habe. Eine Aussage wird dabei auch nicht dadurch richtig, dass mehrere Modelle sie wiederholen. Für mich kommt es deshalb weniger auf die Wahl des Modells an als auf die Prüfung, die man selbst dahinterschaltet.

3.5 KI-Reflexion (Kristian Popov)

Mein Schwerpunkt lag während des Projekts auf den Themen *Endliche Automaten* und *Reguläre Sprachen*. Dabei habe ich Gemini 3.1 Pro, die kostenlose Version von ChatGPT sowie Claude Opus 4.8 und später Opus 5 ausprobiert.

Die Modelle waren dabei unterschiedlich gut. Manche erklärten theoretische Zusammenhänge besser, während andere bei der Implementierung oder Fehlersuche hilfreicher waren. Überwiegend arbeitete ich mit Claude, da die Vorschläge bei Programmieraufgaben häufiger direkt brauchbar waren und ich dadurch weniger nacharbeiten musste.

Im Laufe des Projekts ist mir außerdem aufgefallen, dass die Modelle unterschiedlich viel Aufwand in die Planung einer Antwort stecken. Eine höhere Planungsstufe bedeutet mehr verbrauchte Tokens und längere Wartezeiten, führt aber nicht automatisch zu besseren Ergebnissen. Für kleine Änderungen reichte meist eine niedrige Stufe aus, während komplexere Aufgaben mit mehreren Dateien von einer höheren Stufe profitierten. Deshalb habe ich je nach Aufgabe entschieden, welche Einstellung sinnvoll war.

3.5.1 KI als Co-Developer (Kristian Popov)

Die KI habe ich direkt in die Entwicklungsumgebung eingebunden. Dadurch konnte sie auf Projektdateien zugreifen, Änderungen vorschlagen und diese direkt im Projekt überprüfen.

Besonders hilfreich war sie bei wiederkehrenden Aufgaben wie Refactorings, Boilerplate-Code, Unit-Tests, Internationalisierung und dem Beheben von TypeScript-Warnungen. Bei der Auto-

matenlogik und beim Parsen von Grammatiken konnte sie außerdem schnell passende Testfälle ableiten.

Bei Architekturentscheidungen ließ ich mir zunächst mehrere Lösungsansätze mit ihren Vor- und Nachteilen erklären und traf die Entscheidung anschließend selbst. Jede vorgeschlagene Änderung habe ich überprüft und getestet.

Unterstützt wurde diese Arbeitsweise durch Erweiterungen aus dem offiziellen Plugin-Verzeichnis `claude-plugins-official`. Aktiv waren mehrere, darunter `typescript-lsp` für Typinformationen über das LSP. Für meine Themen am wichtigsten waren zwei:

`playwright`
: ermöglicht der KI, die Anwendung im Browser auszuführen und aus Sicht des Benutzers zu testen. Dadurch konnten beispielsweise Oberflächen, Navigation und Benutzerinteraktionen überprüft werden.

`chrome-devtools-mcp`
: ermöglicht über das MCP den Zugriff auf Konsolenausgaben und das Verhalten der laufenden Anwendung. Dadurch konnte ich bei Fehlern genauer untersuchen, was zur Laufzeit tatsächlich passiert.

Zusätzlich habe ich das Projekt einmalig mit Graphify [1] analysiert und einen Wissensgraphen erzeugt. Dadurch konnte die KI bei späteren Aufgaben gezielt auf relevante Dateien, Klassen und Funktionen zugreifen, ohne das gesamte Projekt erneut einlesen zu müssen.

3.5.2 Automatisierte Aufgabengenerierung (Kristian Popov)

Für die Übungen wurden neben der Aufgabenstellung auch eine Referenzlösung, ein Startprogramm und Beispielwörter benötigt. Die KI nutzte ich vor allem, um Varianten bestehender Aufgaben zu erzeugen, beispielsweise mit anderem Alphabet, anderen Wortlängen oder unterschiedlichen Schwierigkeitsgraden.

Ob eine Aufgabe tatsächlich korrekt war, habe ich anschließend mit der automatischen Übungskontrolle des Simulators gegen die Referenzlösung geprüft (siehe Abschnitt 4.2). Nur geprüfte Aufgaben wurden in den Aufgabenkatalog aufgenommen (siehe Abbildung 2).

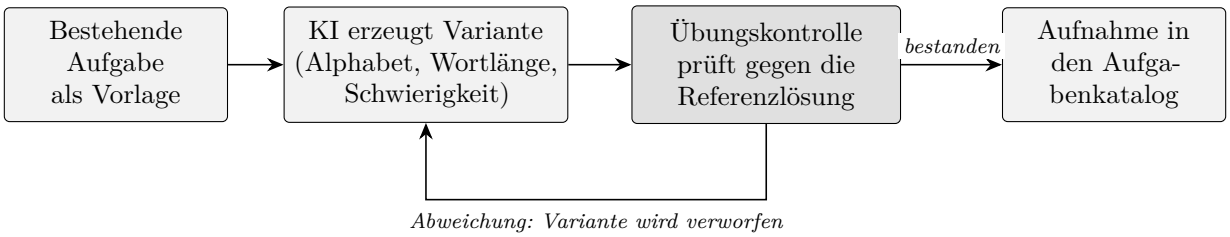


Abbildung 2: Erzeugung und Prüfung von Aufgabenvarianten. Nur Varianten, die sich gegenüber der Referenzlösung identisch verhalten, werden in den Aufgabenkatalog übernommen.

3.5.3 Kritische Reflexion (Kristian Popov)

Die KI lieferte häufig brauchbare, aber nicht immer korrekte Vorschläge. Teilweise verwendete sie APIs, die in der verwendeten Version nicht mehr existierten, oder erfand Funktionen und Bibliotheken. Auch Sonderfälle und Auswirkungen auf andere Teile des Projekts wurden gelegentlich übersehen.

Ein weiteres Problem war die starke Ähnlichkeit zwischen den Themen *Endliche Automaten* und *Reguläre Sprachen*. Statt bestehende Funktionen zu abstrahieren, erzeugte die KI teilweise doppelte Implementierungen. Änderungen, die mehrere Themen gleichzeitig betrafen, musste ich deshalb häufig in kleinere Schritte aufteilen.

Insgesamt sparte mir die KI besonders bei wiederkehrenden Aufgaben und beim Ausbau bestehender Funktionen viel Zeit. Bei theoretischen Konzepten und größeren Architekturänderungen war jedoch weiterhin meine eigene Prüfung entscheidend. Ein Orchestrator wie Sakana Fugu könnte künftig die Auswahl des passenden Modells übernehmen [2]; da das System erst kurz vor Abschluss meiner Arbeit erschien, konnte ich es nicht mehr einsetzen.

Für mich war die KI damit vor allem ein Werkzeug zur Unterstützung. Die fachlichen Entscheidungen und die Verantwortung für die Korrektheit der Implementierung lagen jederzeit bei mir.

4 Designentscheidungen der Fachmodule

4.1 Turingmaschine (Richard Schröder)

Der Themenbereich Turingmaschine umfasst eine Übersicht, sieben Beispiele, dreizehn Übungen und eine freie Simulation. Die folgenden Abschnitte begründen die Entscheidungen für Fachmodell, Editor, Laufzeitdarstellung und Überprüfung.

4.1.1 Entscheidungen in der Architektur (Richard Schröder)

Die Implementierung ist in `domain`, `application`, `presentation` und `adapters` gegliedert. `TMProgram` enthält die editierbare Maschinenkonfiguration mit Zuständen, Alphabeten, Blank-Symbol, Start- und Haltezuständen, Übergängen und Anfangsband. `TMRuntime` hält den aktuellen Zustand, die Kopfposition, den Bandinhalt, den Schrittzähler und die Halteinformation. Gegenüber einem gemeinsamen veränderlichen Modell bleibt die Konfiguration beim Vorwärts- und Rückwärtsschritt unverändert, beim Start muss sie dafür in einen Laufzeitzustand überführt werden.

`TuringMachineSimulator` stellt die Operationen zum Erzeugen des Anfangszustands, Lesen des aktuellen Symbols und Ausführen eines Schritts wie im Interface `ISimulator` gefordert bereit. Die Anwendungsschicht führt eine Historie der Laufzeitzustände für den Rückschritt. Dadurch lassen sich die Zustandsübergänge ohne UI ausführen und prüfen, während die Historie jedoch zusätzlichen Speicher pro Schritt benötigt.

Fehlende Übergänge werden als reguläres Halten und nicht als technischer Fehler behandelt. Ein nicht definierter Übergang erzeugt den Haltegrund `missing_transition`, ein vorgesehener Haltezustand `halt_state`. Dadurch können Lernende unvollständige Übergangstabellen schrittweise bearbeiten und testen. Die Alternative, jeden fehlenden Übergang als Validierungsfehler zu behandeln, würde den Start solcher Zwischenstände verhindern.

Die Lösungskontrolle startet für jeden der Validierungsfälle eine eigene Simulation und verlangt zusätzlich das Ende in einem vorgesehenen Haltezustand. Der Haltezustand ist nötig, weil ein fehlender Übergang in der freien Simulation regulär hält, aber keine gelöste Aufgabe belegt. Neben dem Ergebnis prüft die Ausführung nur bis zu einer Schrittgrenze, damit eine nicht haltende Maschine die Kontrolle nicht unbegrenzt blockiert.

Der Editor verwaltet Zustände, Bandsymbole, Übergänge und Haltezustände in einem Reducer und einem zugehörigen Hook. Beim Umbenennen eines Zustands werden Startzustand, Haltezu-

stände und Zielzustände der Übergänge mitgeführt. Das fachliche Modell speichert die Übergänge als verschachtelte Struktur aus Lesesymbol und Zustand. `tmTableAdapter` überträgt sie erst für die Darstellung in Spalten, Zeilen und die aktive Tabellenzelle. Eine direkte Speicherung im Tabellenformat würde die Domain-Schicht an Zeilen- und Spaltenindizes der UI binden.

Das Eingabealphabet wird aus dem Anfangsband abgeleitet. Das Bandalphabet berücksichtigt das vorhandene Alphabet, das Anfangsband, das Blank-Symbol sowie die Lese- und Schreibsymbole der Übergänge. Gegenüber zwei vollständig manuell gepflegten Alphabeten entfallen doppelte Eingaben.

Für den Transfer wurde eine binäre Kodierung statt einer lesbaren JSON-Darstellung gewählt. Der Turingmaschinen-Codec vergleicht mehrere Darstellungen der Übergangstabelle und übernimmt die kürzeste erzeugte Variante. Die Base64URL-Zeichenkette lässt sich kopieren und im gemeinsamen Training Center speichern, kann aber ohne den passenden Decoder weder gelesen noch bearbeitet werden.

4.1.2 Entscheidungen im UI (Richard Schröder)

Die Simulation beginnt im Konfigurationsmodus. Dort werden Anfangsband, Startzustand, Übergangstabelle, Haltezustände, Alphabete sowie Import und Export bearbeitet. Erst nach dem Anwenden einer gültigen Konfiguration wechselt die Ansicht in den Laufzeitmodus. Bei einem ungültigen Übergangsformat bleibt die betroffene Tabellenzelle editierbar, weitere Änderungen und der Start der Simulation werden bis zur Korrektur gesperrt.

Die Zeilen der Übergangstabelle stehen für Bandsymbole, die Spalten für Nicht-Haltezustände. Haltezustände erhalten keine editierbaren Übergangsspalten, weil das gewählte Maschinenmodell die Ausführung beim Erreichen eines solchen Zustands beendet. Leere Zellen bilden die erlaubte partielle Übergangsfunktion ab. Das Blank-Symbol erscheint als ⋄, die Tastaturform <> wird beim Eingeben in dieses Symbol umgewandelt.

Die Laufzeitansicht zeigt Band, Simulator-Controls, Statuskarte und Übergangstabelle. Die Zahl sichtbarer Bandzellen wird aus der verfügbaren Breite berechnet. Eine Verschiebung wird nur animiert, wenn sich das Fenster um höchstens eine Zelle bewegt und die Schrittdauer über 140 Millisekunden liegt. Die daraus berechnete Animationsdauer ist auf 120 bis 600 Millisekunden begrenzt. Bei schnelleren Schritten oder größeren Sprüngen wird die neue Position ohne Verschiebeanimation dargestellt.

Die gemeinsamen Simulator-Controls bieten Play, Pause, Schritt, Rückschritt, Reset und Geschwindigkeitsregelung. Die Statuskarte nennt Zustand, gelesenes Symbol, Schrittzahl und Haltegrund. Der Tabellenadapter markiert zugleich den Übergang für den aktuellen Zustand und das gelesene Symbol. Übungen ergänzen die Aktionen zum Laden und Prüfen einer Lösung. Der Ergebnisdialog zeigt Eingabe, Ausgabe, erwartete Ausgabe und das Erreichen eines gültigen Haltezustands.

4.1.3 Andere Entscheidungen (Richard Schröder)

Ein Beispiel enthält ein fertiges Programm. Eine Übung ergänzt Starter- und Lösungsprogramm, Speicher-ID, Schwierigkeitsgrad und Validierungsfälle. Die dreizehn Übungen verteilen sich auf fünf einfache, vier mittlere und vier schwere Aufgaben. Sieben Beispiele stellen fertige Maschinen bereit. Derselbe Simulator verarbeitet damit freie Konfigurationen, Beispiele und Übungen.

Die Unit-Tests behandeln unter anderem Domain-Logik, Validierung, Transfer, Editor, Tabellenadapter, Banddarstellung und Lösungskontrolle. Die E2E-Tests prüfen sichtbare Bedienabläufe,

Navigation, Local Storage, Import und Export sowie Fehlervalidierung. `data-testid`-Attribute kennzeichnen die dafür verwendeten UI-Elemente.

4.2 Endliche Automaten (Kristian Popov)

Im Thema *Endliche Automaten* können Studierende deterministische und nichtdeterministische endliche Automaten selbst definieren und Eingabewörter Schritt für Schritt ausführen. Jeder Simulationsschritt wird gleichzeitig in der Übergangstabelle und im Zustandsdiagramm dargestellt. Ergänzt wird die Einzelsimulation durch einen Stapeltest für mehrere Wörter, die Umwandlung rechtslinearer Grammatiken in Automaten und Übungen mit automatischer Rückmeldung.

4.2.1 Entscheidungen in der Architektur (Kristian Popov)

Ursprünglich war nur die Unterstützung deterministischer Automaten vorgesehen. Nichtdeterministische kamen hinzu, weil dort ein Symbol in mehrere Zustände zugleich führen darf: Ein Automat ist so oft direkt aus der Sprachbeschreibung ablesbar und kompakter, und die Mehrdeutigkeit muss erst beim Schritt zum DFA aufgelöst werden. Da jeder DFA gleichzeitig ein NFA ist, wurde für beide Varianten bewusst nur eine Ausführungslogik verwendet. Der Kern arbeitet mit einer Menge aktiver Zustände, die bei einem DFA höchstens ein Element enthält; ε-Übergänge werden dabei stets über die ε-Hülle aufgelöst. Der deklarierte Automatentyp steuert Validierung und Eingabemaske, nicht aber die Ausführung.

Der Simulator ist in fünf Bereiche aufgeteilt (siehe Abbildung 3):

`domain`
: enthält die Fachlogik wie Simulation, Validierung, Grammatikumwandlung, Stapeltest und Übungskontrolle.

`application`
: verbindet die Fachlogik mit React und verwaltet Simulation und Editor.

`adapters`
: bereitet die Daten für die Übergangstabelle auf.

`presentation`
: enthält die Benutzeroberfläche mit Diagramm, Tabelle, Eingabewort, Stapeltest und Grammatikeingabe.

`config`
: enthält Standardautomaten und Beispiele.

Die Trennung sorgt dafür, dass die Fachlogik unabhängig von React bleibt und ohne gerenderte Oberfläche testbar ist. Das Thema *Reguläre Sprachen* folgt demselben Schnitt und greift zusätzlich direkt auf die Grammatikumwandlung dieses Themas zurück.

Ein Simulationsschritt erzeugt einen neuen Zustand mit der bisherigen Historie. Dadurch sind sowohl das Zurückgehen als auch die Auswahl früherer Schritte möglich, ohne eine zusätzliche Historie in der Oberfläche zu verwalten.

Auch die Validierung ist von der Oberfläche getrennt. Sie liefert Fehlercodes und relevante Werte, während die Oberfläche daraus die passenden Meldungen erzeugt. Dadurch bleibt die Fachlogik unabhängig von der Sprache der Benutzeroberfläche.

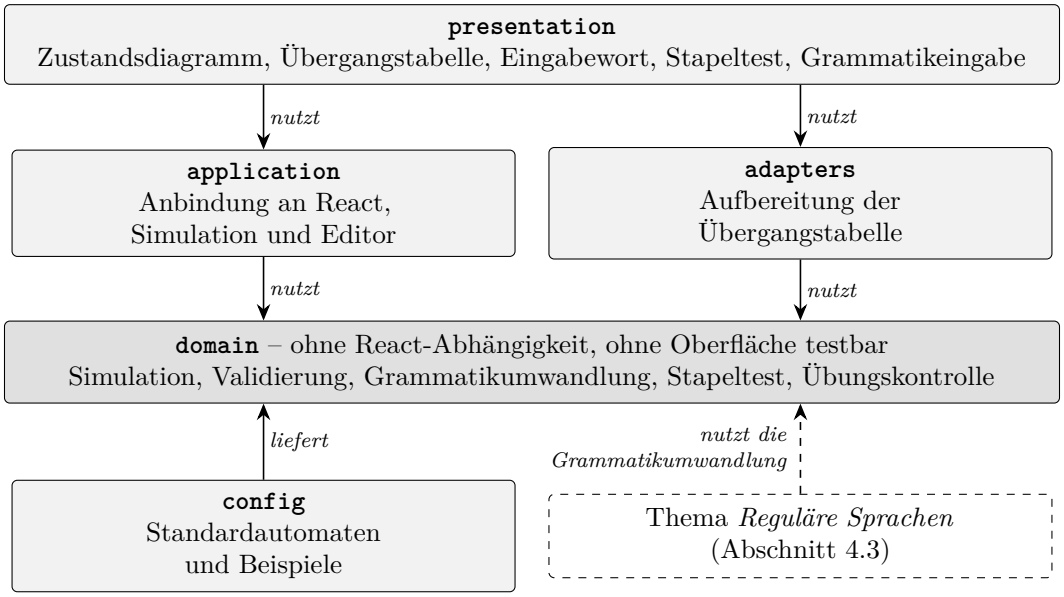


Abbildung 3: Schichten des Simulators für endliche Automaten. Durchgezogene Pfeile zeigen, welche Schicht auf welche zugreift. Die Fachlogik in `domain` kennt React nicht und ist ohne gerenderte Oberfläche testbar; das Thema *Reguläre Sprachen* greift ausschließlich auf diese Schicht zu.

Die mitgelieferten Automaten sind bewusst partiell definiert. Fehlt ein Übergang, wird das Wort abgelehnt. Auf einen zusätzlichen Fangzustand wurde verzichtet, da dieser zwar die Sprache nicht verändert, aber die Diagramme unnötig vergrößert.

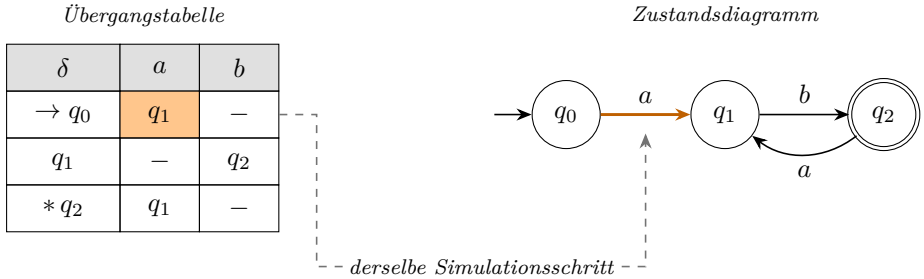
4.2.2 Entscheidungen im UI (Kristian Popov)

Ursprünglich war für die Simulation nur die Übergangstabelle vorgesehen. Während der Entwicklung zeigte sich jedoch, dass der Ablauf eines Wortes damit schwer nachzuvollziehen ist: Zyklen und Sackgassen werden erst sichtbar, wenn man mehrere Zeilen gedanklich verknüpft. Deshalb wurde zusätzlich ein Zustandsdiagramm integriert.

Beide Darstellungen sind an denselben Simulationsschritt gebunden (siehe Abbildung 4): Die Tabelle hebt genau die Zelle hervor, die dem im Diagramm markierten Übergang entspricht. Sie zeigt die formale Definition des Automaten, während das Diagramm den tatsächlichen Weg des Wortes sichtbar macht. Bei größeren Automaten wird das automatisch berechnete Layout schnell unübersichtlich. Einzelne Zustände lassen sich deshalb frei ziehen, der Ausschnitt vergrößern und verkleinern und die Ausgangsansicht mit einem Klick wiederherstellen. Aktive und fehlerhafte Übergänge bleiben dabei auch bei geringer Zoomstufe beschriftet.

Verstöße gegen den Determinismus werden nicht nur aufgelistet, sondern unmittelbar an Zustand und Übergang im Diagramm markiert. Damit ist die Ursache dort sichtbar, wo sie entsteht.

Der Editor umfasst alle wesentlichen Bestandteile eines Automaten: Automatentyp, Alphabet, Zustände, Start- und Endzustände sowie das Eingabewort. Die Zustandsmenge wird direkt über die Übergangstabelle verwaltet. Alphabetsymbole sind auf einzelne Zeichen beschränkt, damit eine Eingabe wie `10` nicht versehentlich als ein einzelnes Symbol aufgenommen wird. Die Zeile für ε-Übergänge erscheint nur bei einem NFA, sobald sie belegt ist oder ausdrücklich angefordert wird.



δ	a	b
$\rightarrow q_0$	q_1	–
q_1	–	q_2
$*\,q_2$	q_1	–

Abbildung 4: Beim Lesen des Symbols a im Zustand q_0 hebt die Übergangstabelle genau die Zelle hervor, die dem im Zustandsdiagramm markierten Übergang entspricht. Der Automat ist partiell definiert; ein fehlender Übergang ist mit – gekennzeichnet und führt zur Ablehnung des Wortes.

4.2.3 Andere Entscheidungen (Kristian Popov)

Neben der Einzelsimulation gibt es einen Schrittverlauf, über den bereits erreichte Schritte direkt ausgewählt werden können. Der gewählte Schritt wird dabei vom Anfang des Laufs an neu berechnet statt zwischengespeichert; bei den hier üblichen Wortlängen ist das unkritisch und hält Anzeige und tatsächlichen Ablauf zwingend deckungsgleich. Der Stapeltest ermöglicht es, mehrere Wörter gleichzeitig zu prüfen. Ein Ergebnis kann anschließend direkt in die Einzelsimulation übernommen werden.

Eine weitere Funktion ist die Umwandlung rechtslinearer Grammatiken in Automaten. Nichtterminale werden dabei zu Zuständen und Produktionen werden in entsprechende Übergänge oder Endzustände übersetzt. Für die Eingabe werden zwei Schreibweisen unterstützt, die automatisch erkannt werden.

Beispiele und Übungen sind getrennt aufgebaut. Beispiele enthalten fertige Automaten und dienen zum Nachvollziehen, während Übungen mit einem leeren Automaten beginnen und zusätzlich Aufgabenstellung, Referenzlösung und automatische Kontrolle enthalten. Die Übungen sind nach *Sprache* → *Automat* und *Grammatik* → *Automat* gegliedert.

Die Übungskontrolle vergleicht nicht die Struktur der Automaten, sondern deren Verhalten. Dazu werden alle Wörter über dem gemeinsamen Alphabet bis zu einer festgelegten Länge geprüft. Der Automat der Studierenden muss für jedes Wort dieselbe Entscheidung treffen wie die Referenzlösung. Bei einer Abweichung wird das erste unterschiedliche Wort als Gegenbeispiel angezeigt. Dadurch können auch unterschiedlich aufgebaute, aber funktional korrekte Automaten akzeptiert werden. Die Prüfung stellt jedoch keinen formalen Äquivalenznachweis dar. Strukturelle Mängel wie ungültige Übergänge werden vorrangig gemeldet, da sie meist die eigentliche Ursache sind und eine Liste fehlgeschlagener Testwörter dann in die Irre führen würde.

Der Bearbeitungsstand jeder Übung wird gespeichert und beim erneuten Öffnen wiederhergestellt, sodass der Wechsel zwischen Übungen keine bereits begonnene Bearbeitung überschreibt.

4.3 Reguläre Sprachen (Kristian Popov)

Das Thema *Reguläre Sprachen* unterstützt die Arbeit mit regulären Sprachen und rechtslinearen Grammatiken. Es bietet drei miteinander verbundene Funktionen: die Aufzählung erzeugter Wörter, die Erzeugung einer Grammatik aus einer Sprachbeschreibung und die Prüfung einzelner Wörter. Sprachbeschreibungen können als Aufzählung oder in Mengenschreibweise eingegeben werden. Die Aufzählung ist dabei auf eine maximale Wortlänge und Wortanzahl begrenzt. Beide

Umwandlungsrichtungen sind zusätzlich als Übungen mit automatischer Rückmeldung verfügbar.

4.3.1 Entscheidungen in der Architektur (Kristian Popov)

Der Simulator besteht aus einer koordinierenden Komponente und fachlich getrennten Bausteinen. `RLSimulator` verwaltet die drei Arbeitsmodi und den Einstieg in Übungen, enthält selbst aber keine formale Logik. Diese liegt im Verzeichnis `domain` und bleibt dadurch unabhängig von React testbar.

Die Fachlogik ist in mehrere Bereiche aufgeteilt (siehe Abbildung 5):

Grammatikverarbeitung
Parser und Validierung prüfen rechtslineare Grammatiken und überführen sie in gemeinsame Datentypen.

Sprachbeschreibung
Ein eigener Parser verarbeitet Aufzählungen und Mengenschreibweisen und erzeugt daraus eine strukturierte Beschreibung.

Umwandlung und Vereinfachung
Sprachbeschreibungen werden in strikt rechtslineare Grammatiken umgewandelt. Gleichwertige Nichtterminale können anschließend zusammengefasst werden.

Sprachauswertung
Die Aufzählung und Worterkennung sowie die Übungskontrolle verwenden dieselben fachlichen Funktionen.

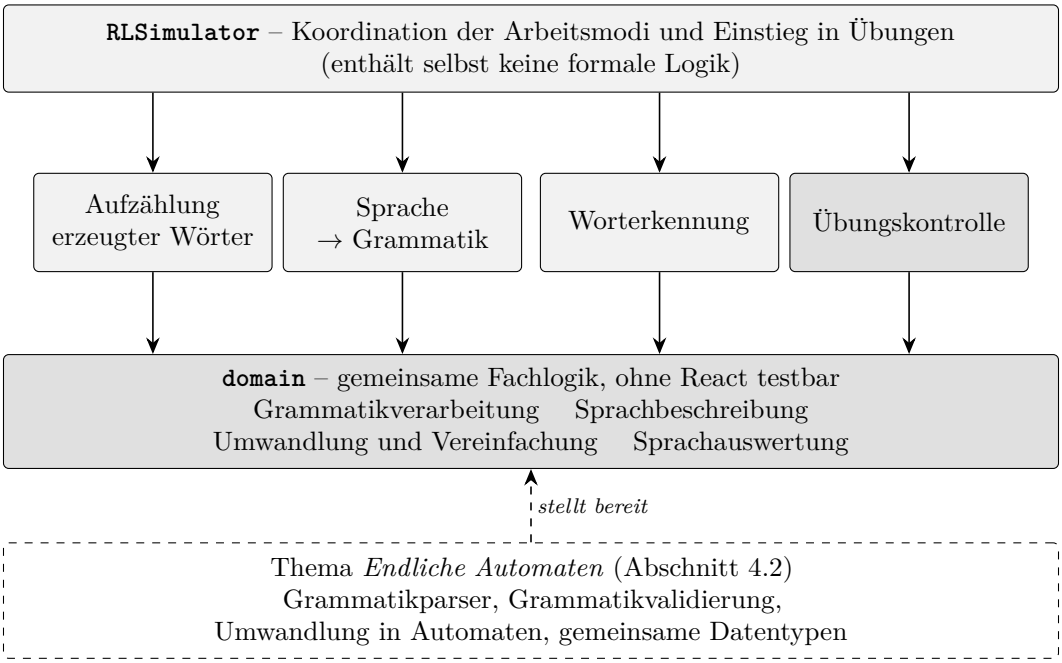


Abbildung 5: Aufbau des Simulators für reguläre Sprachen. Alle drei Arbeitsmodi und die Übungskontrolle greifen auf dieselbe Fachlogik zu. Dadurch stimmen Aufzählung, Worterkennung und Übungskontrolle stets überein.

Wiederverwendet werden der Grammatikparser, die Grammatikvalidierung und die gemeinsamen Datentypen. Eine eigene Ableitungslogik direkt auf den Produktionen hätte zwei Implementierungen erfordert, die dasselbe Verhalten abbilden müssen; mit der gemeinsamen Grundlage stimmen Aufzählung, Worterkennung und Übungskontrolle stets überein.

Der Parser unterstützt sowohl direkte Aufzählungen, beispielsweise `L = {"010", "0010"}`, als auch Mengenschreibweisen wie `L = {a^n c b^m | n, m > 0}`. Endet eine Aufzählung mit einer Fortsetzungsmarkierung, versucht der Parser daraus ein wiederkehrendes Muster abzuleiten, statt die Wörter nur wörtlich zu übernehmen. So lässt sich eine unendliche Sprache angeben, ohne die Mengenschreibweise schon zu beherrschen.

Bestimmte Eingaben werden bewusst abgelehnt, auch wenn sie sich formal beschreiben lassen. Verwendet dieselbe Variable mehrere Faktoren, wie in `a^n b^n`, koppelt das deren Längen aneinander; eine solche Sprache ist nicht regulär. Obere Schranken werden ebenfalls nicht unterstützt, obwohl sie endliche und damit reguläre Sprachen beschreiben, denn sie lassen sich klarer als Aufzählung angeben. Wiederholungen innerhalb einer Gruppe, etwa `(a^n)b`, weist der Parser zurück und verweist auf die gleichwertige Schreibweise `a^n b`. Diese Fälle ausdrücklich zu erklären, ist lehrreicher, als sie stillschweigend zu verwerfen.

Die erzeugten Grammatiken bleiben strikt rechtslinear und verwenden ausschließlich Produktionen der Form $A \to aB$, $A \to a$ und $A \to \varepsilon$. Eine kompaktere, aber allgemeinere kontextfreie Darstellung wurde bewusst nicht verwendet, da sie die Grenze zu anderen Grammatiktypen überschreiten würde. Für die Ausgabe gibt es zwei Sichten auf dasselbe Ergebnis: Die detaillierte Sicht zeigt ein Nichtterminal je Präfix, die vereinfachte führt Nichtterminale zusammen, die dieselben Wörter ableiten. Die Vereinfachung bleibt rechtslinear und kann die Grammatik nur verkleinern, nie ihre Sprache verändern.

Alle Stellen, die eine rechtslineare Grammatik ausgeben, benutzen dieselbe Namensvergabe und Ausgabefunktion. Zuvor lag diese Logik in drei leicht unterschiedlichen Kopien vor; in einer davon verbrauchte ein weggelassener Endzustand einen Buchstaben, sodass in der Ausgabe etwa `E` übersprungen wurde und eine exportierte Grammatik nach dem Import andere Namen trug.

4.3.2 Entscheidungen im UI (Kristian Popov)

Die drei Arbeitsweisen sind über eine Modusauswahl erreichbar. Die Komponenten bleiben beim Wechsel zwischen den Modi erhalten, sodass begonnene Eingaben nicht verloren gehen.

Bei geöffneten Übungen wird eine geführte Arbeitsfläche angezeigt, in der Vorgabe und Aufgabe nebeneinander stehen. Ein Wechsel des Modus würde den aktuellen Bearbeitungsstand verwerfen und erfordert deshalb eine Bestätigung.

Fehlermeldungen erscheinen direkt am jeweiligen Eingabefeld und erst nach einer Eingabe. Zusätzlich kann die unterstützte Syntax direkt am Eingabefeld angezeigt werden.

4.3.3 Andere Entscheidungen (Kristian Popov)

Die Übungskontrolle vergleicht nicht die Texte der Lösungen, sondern deren Verhalten. Ein Textvergleich mit einer Musterlösung wäre unbrauchbar, da dieselbe Sprache von vielen unterschiedlichen Grammatiken erzeugt werden kann und die Sprachen zum Teil unendlich sind.

Bei *Sprache → Grammatik* prüft die Kontrolle zwei Dinge. Zunächst muss die eingegebene Grammatik die geforderten Wörter erzeugen und die ausgeschlossenen Wörter ablehnen. Anschließend wird sie mit der Referenzgrammatik verglichen: Beide werden in Automaten überführt und gemeinsam durchlaufen, bis ein Wort auftritt, das nur eine der beiden Sprachen enthält. Ein solches Wort erscheint als Gegenbeispiel in der Rückmeldung. Dadurch werden unterschiedlich aufgebaute Grammatiken akzeptiert, solange sie dieselbe Sprache erzeugen; eine Grammatik, die nur die vorgegebenen Beispielwörter richtig behandelt, genügt dagegen nicht. Der gemeinsame Durchlauf bricht nach einer festen Schranke ab und liefert dann kein abschließendes Urteil; die Grammatiken der Übungen erreichen diese Schranke nicht.

Bei *Grammatik → Sprache* verwendet die Kontrolle dieselben Grenzen für Wortlänge und Wortanzahl wie die Anzeige der Lösung. Andernfalls könnte eine angezeigte Lösung die anschließende Prüfung nicht bestehen. Die Rückmeldung zeigt auch hier konkrete fehlende oder falsch erzeugte Wörter statt nur einer Ja-Nein-Aussage. Diese Richtung bleibt damit eine Stichprobe im festgelegten Bereich und kein Nachweis über die gesamte Sprache.

Für Import und Export werden unterschiedliche Formate verwendet. Grammatiken werden strukturiert gespeichert, während Sprachbeschreibungen als Text erhalten bleiben. Die Daten werden beim Kodieren und Dekodieren auf ihre Gültigkeit geprüft.

Jeder Modus speichert seinen Zustand getrennt in der lokalen Ablage, sodass sich die Eingaben der verschiedenen Arbeitsbereiche nicht gegenseitig überschreiben. Die Aufgabenstellungen verwenden außerdem eine sprachneutrale Notation und müssen daher nicht in den Übersetzungsdateien gepflegt werden.

4.4 Zahlensysteme (Enrico Ebert)

Ziel des Moduls ist es, abstrakte Abläufe wie Bit-Transformationen, Komplementbildungen und Arithmetik interaktiv und schrittweise begreifbar zu machen. Im Folgenden werden die wichtigsten architektonischen, visuellen und organisatorischen Entscheidungen der Umsetzung dokumentiert.

4.4.1 Anforderungen und Herausforderungen (Enrico Ebert)

Die Umsetzung von Zahlensystemen stellte uns vor mehrere Herausforderungen. Zum einen mussten wir die mathematische Korrektheit bei großen Zahlen gewährleisten, zum anderen sollten die Zwischenschritte für Studierende nachvollziehbar sein. Die Umrechnung zwischen verschiedenen Basen, die Darstellung negativer Zahlen im Zweierkomplement und die Erklärung von IEEE-754-Gleitkommazahlen erforderten jeweils unterschiedliche didaktische Ansätze.

4.4.2 Architektur (Enrico Ebert)

Die Architektur des Simulators unterteilt sich in fünf Modi: *integers*, *negative*, *fractional*, *ieee754* und *arithmetic*. Diese Trennung ermöglichte es uns, jeden Modus unabhängig zu entwickeln und zu testen. Ein zentrales Zustandsobjekt dient dabei als *Single Source of Truth*. Beim Start ordnet eine Hilfsfunktion die ausgewählte Übung oder das Beispiel automatisch dem passenden Modus zu.

Für alle mathematischen Berechnungen verwenden wir JavaScripts BigInt statt `Number`. Die Entscheidung für BigInt ergab sich aus der Erfahrung, dass JavaScripts reguläre Number-Typ bei großen Ganzzahlen an Präzision verliert. So vermeiden wir zuverlässig Rundungsfehler und ungewollte Überläufe bei Zahlen jenseits des sicheren Integer-Bereichs.

Eingaben werden je nach Basis mit strengen Regex-Filtern validiert. Wir begrenzen die Bitlänge explizit, um Speicherüberläufe zu verhindern. Bei negativen Zahlen arbeiten wir mit einer festen Breite von 8 Bit, was einen Wertebereich von -128 bis 127 abdeckt. Diese Einschränkung folgt dem Vorlesungsstoff und vereinfacht die didaktische Darstellung.

Ein technischer Kniff kommt bei der IEEE-754-Darstellung zum Einsatz: Um die exakte Bit-Repräsentation einer Gleitkommazahl zu erhalten, schreiben wir sie in ein `Float32Array` und lesen sie über ein `Uint32Array` wieder aus. Dieser Ansatz nutzt die nativen Float32-Operationen des Browsers und vermeidet manuelle Bit-Manipulationen. Spezielle Zustände wie *NaN* oder ±Infinity fangen wir ab und erklären sie gesondert in Informationskarten.

Die arithmetischen Basisoperationen sind modular in eigenen Pipelines gekapselt. Die Addition nutzt einen Volladdierer, die Multiplikation den Shift-and-Add-Algorithmus. Diese Aufteilung ermöglichte es uns, jeden Operator einzeln zu implementieren und zu testen, bevor sie in den Gesamtsimulator integriert wurden. Jeder Durchlauf generiert strukturierte Schritt-Objekte, die direkt mit den i18n-Schlüsseln für die Lokalisierung verknüpft sind.

4.4.3 UI-Entscheidungen (Enrico Ebert)

Das Interface ist stark auf die Didaktik ausgerichtet. Wir entschieden uns bewusst für einen Step-by-Step-Ansatz, bei dem die ersten Rechenschritte standardmäßig aufgeklappt sind. Dies erleichtert den Einstieg, da Studierende sofort sehen, wie ein Ergebnis zustande kommt.

Die Darstellung passt sich dem jeweiligen Modus an:

- **Wandler (Ganz-/Kommazahlen):** Wir nutzen klassische, statische Mathematik-Blöcke, die an Divisionstabellen und Summenformeln aus der Schule erinnern. Die vertraute Darstellung hilft beim Verständnis der Umrechnungslogik.
- **Arithmetik:** Die Operationen werden als schriftliches Rechengitter nachgebildet. Die Bits für Operanden, Carry und Summe sind farbcodiert, sodass der Informationsfluss visuell erkennbar wird. Bei einem Overflow warnt ein dediziertes Banner, das auf den Überlauf aufmerksam macht.
- **Negative Zahlen:** Um den Vorlesungsstoff abzubilden, werden Vorzeichen-Betrag, Einer- und Zweierkomplement direkt nebeneinander visualisiert. Interaktive Zeilen steuern die jeweilige Erklärung, sodass Studierende selbst wählen können, welcher Schritt vertieft werden soll.
- **IEEE-754:** Die 32 Bits sind visuell in Sign, Exponent und Mantisse unterteilt. Informationskarten erklären die Normalisierung und den Bias, was das Verständnis der IEEE-754-Darstellung erleichtert.

4.4.4 Organisation und State Management (Enrico Ebert)

Die Lerninhalte bestehen aus 10 nach Schwierigkeit sortierten Übungen und 35 vorlesungsbegleitenden Beispielen. Wir haben uns für diese Struktur entschieden, um Studierenden einen klaren Progressionspfad zu bieten. Als ID fungiert der `titleKey`, wodurch der Lernfortschritt auch bei einem Sprachwechsel im i18n-System erhalten bleibt. Beispiele zeigen sofort die Lösung und fließen nicht in den Fortschritt ein, da sie primär der Erklärung dienen.

Der Anwendungszustand wird alle 10 Sekunden sowie beim Verlassen der Komponente automatisch lokal gesichert. Diese Entscheidung minimiert Datenverluste bei versehentlichen Tab-Schließungen. Übungen und der freie Simulator nutzen dabei getrennte Speicherslots, um Konflikte zu vermeiden.

4.5 Moore- und Mealy-Automaten (Markus Nöth)

Das Modul Moore- und Mealy-Automaten vermittelt den Entwurf und die Funktionsweise synchroner Schaltwerke über interaktive Tabellen, Zustandsgraphen, Logikschaltungen und Übergangstabellen. Es umfasst freie Simulatoren für beide Automatentypen sowie strukturierte Beispiele und Übungsaufgaben für Moore- und Mealy-Maschinen.

4.5.1 Entscheidungen in der Architektur (Markus Nöth)

Die Implementierung beider Automatentypen baut auf dem gemeinsamen Interface `ISimulator<TConfig,TState>` der Plattform auf. Die mathematische Definition und die Aufgabengestaltung orientieren sich an den Vorlesungsfolien von Prof. Dr. Braun sowie am Lehrbuch von Vossen und Witt [3, Kapitel 4]: Beim Moore-Automaten hängt die Ausgabe nur vom aktuellen Zustand ab ($\lambda : S \to \Gamma$). Beim Mealy-Automaten ist die Ausgabefunktion an das Wertepaar aus aktuellem Zustand und anliegendem Eingabesymbol gekoppelt ($\lambda : S \times \Sigma \to \Gamma$).

Die Simulationslogik kapseln die Klassen `MooreMachineSimulator` und `MealyMachineSimulator`. Bei jedem Taktschritt sichern sie den gesamten Ausführungszustand in einer Historie ab. Dadurch kann der Benutzer über `stepBack()` deterministisch zurückspringen, ohne dass Berechnungen neu aufgerollt werden müssen oder Seiteneffekte entstehen.

Für beide Automatentypen erzeugt das Modul Wahrheitstabellen, disjunktive Normalformen (DNF) und Logikgatter:

- **Moore-Transformation (`buildMooreConversion`):** Da die Ausgabe allein an den Zustand gekoppelt ist, hängen die Ausgangsvariablen (y) nur von den Zustandsvariablen ($S_0 \dots S_n$) ab. Die Übergangsfunktionen für den Folgezustand ($T_0 \dots T_n$) berechnen sich aus den Zustands- und Eingangsvariablen ($E_0 \dots E_m$).
- **Mealy-Transformation (`buildMealyConversion`):** Sowohl der Folgezustand als auch die Ausgänge hängen gleichzeitig von Zustand und Eingabe ab. Die Wahrheitstabelle der Ausgänge umfasst daher Zeilen mit der Kombination aller Bits von S und E.

Aus den generierten Wahrheitstabellen berechnet das angebundene DNF-Modul über den Funktionsaufruf `computeSmallestBooleanExpression()` die minimierten Schaltfunktionen. Um Blockaden des Browser-Threads durch die exponentielle Rechenkomplexität bei vielen Eingängen auszuschließen, ist die automatische Minimierung auf maximal acht Eingangsvariablen ($2^8 = 256$ Zeilen) begrenzt. Bei größeren Schaltungen gibt das System stattdessen die kanonische Normalform aus.

4.5.2 Entscheidungen im UI (Markus Nöth)

Die Konfiguration der Automaten erfolgt über Editoren, deren Erscheinungsbild für beide Typen bewusst einheitlich gehalten ist:

- **Editor-Tabellen:** Bei Mealy-Automaten werden Folgezustand und Ausgabesymbol gemeinsam in einer Matrixzelle editiert. Bei Moore-Automaten sind Übergangsmatrix und Zustandsausgabetabelle in zwei getrennte Bereiche aufgeteilt.
- **Transformationsansichten:** In der Konvertierungsansicht (`AutomataConversionView`) kann der Anwender zwischen Übergangs- und Ausgabetabellen wechseln. Jede Zeile schlüsselt die symbolischen Werte in ihre Bitbelegung auf. Die zugehörige Formelliste zeigt kanonische und minimierte Ausdrücke für jedes Bit an.
- **Zustandsgraph (`StateMachineGraph`):** Diese Komponente visualisiert den Automaten als gerichteten Graphen. Sie berechnet eine radiale Knotenverteilung, bündelt parallele Kanten mit symmetrischem Versatz und zeichnet Selbstschleifen als Kurven. Zoom und Pan sind über die SVG-`viewBox` implementiert, was Skalierungsartefakte im Vergleich zu CSS-Transformationen vermeidet. Während der Simulation heben sich der aktive Zustand und die geschaltete Kante synchron hervor.
- **Logikgatter-Diagramm (`LogicGateDiagram`):** Stellt die minimierten Schaltfunktionen

für jedes Bit mit standardisierten Gattersymbolen (AND, OR, NOT) dar. Ein Klick auf ein Ausgangsbit hebt den zugehörigen Signalpfad farblich hervor.

Beide Komponenten wurden modular unter `app/src/components/shared/` abgelegt, sodass sie neben Moore- und Mealy-Maschinen auch von künftigen Simulatoren der Plattform direkt wiederverwendet werden können.

4.5.3 Andere Entscheidungen (Markus Nöth)

Das Modul enthält elf praxisnahe Übungsaufgaben: sieben für Moore-Automaten (u. a. Drehkreuz, Aufzug, Ampelsteuerung, Scheibenwischer und Münzprüfer) sowie vier für Mealy-Automaten (u. a. Flankendetektor, Türsteuerung und Sequenzerkennung). Jede Aufgabe stellt ein Starterprogramm, eine Musterlösung und automatisierte Validierungsfälle bereit.

Für Speicherung und Austausch kommt der Modul-Codec `programTransfer.ts` zum Einsatz: Anstelle von unkomprimiertem JSON serialisieren ein `BitWriter` und `BitReader` Zustände, Symbole, Übergänge und Ausgaben direkt in dichte Bitfolgen. Diese werden Base64URL-kodiert und ermöglichen kompakte URLs für das Teilen von Automaten sowie die persistente Ablage im Local Storage des Trainingscenters.

4.6 Disjunktive Normalform (Duy Tan Pham)

Die DNF ist ein eigener Themenbereich im Modul „Grundlagen der Informatik“. Sie folgt dem gleichen Aufbau wie die anderen Themen: Übersicht, Übungen, Beispiele und eine freie Simulation. Die Inhalte bauen schrittweise aufeinander auf: von der kanonischen DNF über die minimale DNF bis zur optional global kleinsten Formel. Die einzelnen Schritte der Ableitung werden dabei sichtbar gemacht.

Die inhaltliche Ausrichtung folgt den vorhandenen Übungsunterlagen der Lehrveranstaltung. Dort wird die DNF aus Wahrheitstabellen mit drei und vier Variablen gebildet. Der Modus „Wahrheitstabelle nach DNF“ setzt genau diesen Aufgabentyp interaktiv um. Die Grenze von vier Variablen deckt damit zugleich die vorgesehenen Übungsbeispiele ab.

4.6.1 Entscheidungen in der Architektur (Duy Tan Pham)

Abbildung 6 fasst den fachlichen Datenfluss der beiden Eingabemodi zusammen.

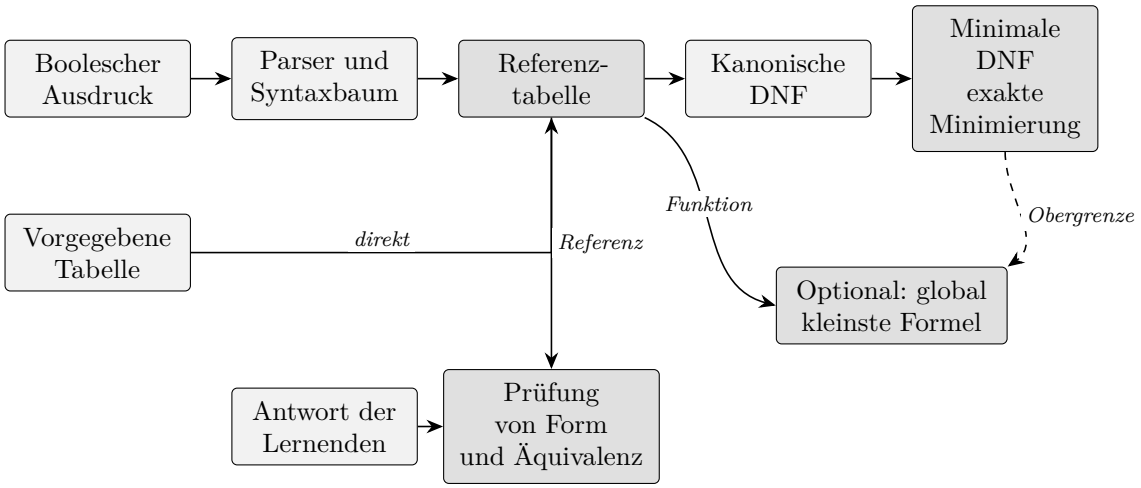


Abbildung 6: Fachlicher Datenfluss im DNF-Bereich. Die Referenztabelle verbindet beide Eingabemodi mit Ableitung, Minimierung, globaler Suche und Äquivalenzprüfung.

- Der Bereich trennt `domain`, `application`, `adapters` und `presentation`. Parser, Wahrheitstabelle, DNF-Ableitung, Minimierung und Prüfung liegen in `domain` und lassen sich ohne React testen.
- Der boolesche Ausdruck wird von einem eigenen Parser (`dnfParser.ts`) in einen Syntaxbaum umgewandelt. Aus diesem Baum wird die Wahrheitstabelle erzeugt. Die Wahrheitstabelle ist die gemeinsame Grundlage für alle weiteren Schritte. Der Parser erlaubt bewusst mehrere Schreibweisen (`!`, `&`, `|` und ebenso ¬, ∧, ∨) und meldet Fehler mit Position.
- Die Ableitung trennt drei Ergebnisstufen sauber: die kanonische DNF (`dnfDerivation.ts`, ein Vollminterm je Eins-Zeile [4, S. 163]), die minimale DNF (`quineMcCluskey.ts`) und die optional global kleinste Formel (`minimalExpression.ts`). Berechnung und Darstellung bleiben dabei getrennt.
- Die wichtigste fachliche Entscheidung betrifft die Minimierung. Die erste Version wählte die restliche Überdeckung nach Quine-McCluskey [4, S. 172] *greedy* aus und konnte dadurch die beste Lösung verpassen. Quine-McCluskey bildet nun die Primimplikanten. Die Funktion `selectCover` wählt daraus mit Branch-and-Bound exakt eine vollständige Abdeckung mit minimaler Literalzahl. Zweige, die das beste bekannte Ergebnis nicht mehr verbessern können, werden frühzeitig verworfen.
- Für die optional kleinste Formel wird zusätzlich eine exakte Synthese über den Funktionsraum berechnet. Die minimale DNF liefert dafür eine gültige Obergrenze. Für Ausdrücke aus Literalen, die mit UND und ODER verbunden werden, findet die Suche bis vier Variablen das globale Literalminimum. Negationen stehen dabei nur direkt an Variablen. Da die Anzahl möglicher Funktionen mit 2^{2^n} stark wächst, ist das Werkzeug auf vier Variablen begrenzt. Diese Grenze wird in Anwendungsschicht, Tabellen-Eingabe und Import geprüft.
- Die Prüfung ist von der Darstellung getrennt. Ein eingegebener Ausdruck wird für jede Zeile der Wahrheitstabelle ausgewertet und nicht durch einen reinen Textvergleich bewertet. Dadurch erkennt die Übungsprüfung (`dnfExerciseCheck.ts`) korrekte, nur äquivalente und nicht äquivalente Antworten. Die Vereinfachungsprüfung (`dnfSimplificationCheck.ts`) prüft zusätzlich, ob die Literalzahl das exakte Minimum erreicht.
- Import, Export und automatisches Speichern verwenden den gemeinsamen Training-Center-Speicher und einen kompakten, URL-sicheren Base64-Code.

4.6.2 Entscheidungen im UI (Duy Tan Pham)

- Der DNF-Bereich bietet zwei Aufgabenmodi: „Ausdruck nach Wahrheitstabelle“ und den umgekehrten Weg „Wahrheitstabelle nach DNF“. Beim zweiten Modus stellen die Lernenden aus einer gegebenen Tabelle die DNF selbst auf. Beide Übungen nutzen dieselben Bausteine.
- Die einzelnen Schritte der Ableitung werden sichtbar gemacht: von den Eins-Zeilen der Tabelle über die passenden kanonischen Terme bis zur minimalen Form. Bei der Vereinfachung wird nicht nur das Ergebnis gezeigt, sondern auch der Lösungsweg. In den Übungen bearbeiten die Lernenden Wahrheitstabellen oder DNF-Ausdrücke selbst und erhalten eine fachlich berechnete Rückmeldung.
- Kleinere Wahrheitstabellen werden vollständig angezeigt. Bei `16` Zeilen, also bei vier Variablen, gibt es eine kompakte, scrollbare Ansicht, die sich auf eine Vollansicht umschalten lässt. Dadurch bleibt die maximale Darstellungshöhe begrenzt.

- Folgeaktionen bleiben sichtbar und werden bis zum Erfüllen ihrer Voraussetzungen ausgegraut. Nur kontextabhängige Schaltflächen, etwa für die Vollansicht einer großen Tabelle, erscheinen bei Bedarf. Dadurch bleibt die Bedienung vorhersehbar.
- Eingaben werden vereinheitlicht: Lernende dürfen `!`, `&` und `|` tippen, angezeigt wird aber immer ¬, ∧, ∨. Das macht die Eingabe leichter, ohne die Darstellung uneinheitlich zu machen.
- Der DNF-Bereich nutzt den gemeinsamen Themenaufbau mit Kopfbereich, Tabs und globaler Sidebar. Dadurch fügt er sich in die übrige Lernplattform ein.

4.6.3 Andere Entscheidungen (Duy Tan Pham)

Die kanonische und die minimale DNF werden bewusst getrennt dargestellt. In der kanonischen DNF enthält jeder Minterm alle Variablen. Eine minimale DNF kann dagegen kürzere Terme besitzen, etwa $(a \wedge c) \vee (b \wedge c)$. Die getrennte Darstellung macht diesen fachlichen Unterschied nachvollziehbar und verhindert, dass eine korrekte Vereinfachung fälschlich als unvollständig angesehen wird.

4.7 Encoding (Enrico Ebert)

Ziel des Encoding-Moduls ist es, die Transformation zwischen unstrukturiertem Text, Codepoints und binären Byte-Sequenzen transparent zu machen. Wir haben die praxisrelevanten Standards UTF-8 und Base64 vollständig implementiert. Der Fokus liegt darauf, die mathematischen und bitweisen Zwischenschritte, wie Bitmasken bei UTF-8 oder das Padding bei Base64, für Studierende visuell nachvollziehbar darzustellen.

4.7.1 Anforderungen und Herausforderungen (Enrico Ebert)

Die Implementierung von Encoding-Verfahren stellte uns vor die Herausforderung, komplexe Byte-Operationen für Studierende verständlich zu machen. UTF-8 und Base64 sind in der Praxis allgegenwärtig, doch die internen Abläufe bleiben oft undurchsichtig. Wir wollten einen Simulator schaffen, der jeden Zwischenschritt einzeln sichtbar macht und so das Verständnis für die zugrundeliegenden Algorithmen fördert.

4.7.2 Architektur (Enrico Ebert)

Wir haben die Architektur in drei klar getrennte Schichten unterteilt: Datenmodell, Transformationslogik und UI. Diese Trennung ermöglichte es uns, die Logik unabhängig von der Darstellung zu entwickeln und zu testen. Der Zustand wird zentral in der Simulator-Komponente verwaltet. Child-Komponenten besitzen keinen eigenen Zustand für verarbeitete Daten, sondern geben Änderungen über State Lifting an die übergeordnete Komponente weiter.

Ein technisches Detail verdient besondere Erwähnung: Um beim Wechsel von Übungen einen sauberen Zustand ohne Artefakte zu garantieren, erzwingen wir über dynamische `key`-Attribute gezielte React-Remounts. Diese Technik stellte sich als notwendig heraus, da reguläres State-Clearing manchmal zu Race Conditions bei schnellen Übergängen führte.

Die UTF-8-Logik nutzt konsequent `Array.from()` zur sicheren Iteration über Unicode-Codepoints. Diese Entscheidung begründet sich darin, dass JavaScripts String-Iteration über Surrogate-Pairs stolpern kann, was zu fehlerhaften Ergebnissen bei emoji und anderen 4-Byte-Zeichen führt. Der Decoder validiert Byteströme streng und weist Sicherheitsrisiken wie *Overlong Encodings*

konsequent ab. Solche Encodings würden mehr Bytes vorschützen als nötig und stellen ein Sicherheitsrisiko dar.

Die Base64-Pipeline arbeitet iterativ in vier Stufen: ASCII-Konvertierung, 8-Bit-Verkettung, 6-Bit-Aufteilung und abschließendes Padding. Wir haben uns für diese schrittweise Darstellung entschieden, damit Studierende sehen können, wie die ursprünglichen 8-Bit-Bytes in die 6-Bit-Gruppen von Base64 zerlegt werden.

4.7.3 Übungsmodus (Enrico Ebert)

Wir haben uns dagegen entschieden, eine statische Liste von Übungen zu pflegen. Stattdessen werden Übungen programmgesteuert dynamisch generiert. Der Generator erzeugt pro Schwierigkeitsgrad Aufgaben inklusive der kanonischen Lösung. Für den Schwierigkeitsgrad *easy* beschränken wir uns auf reines ASCII, für *medium* nutzen wir 2-Byte-Unicode, und für *hard* kommen 3-Byte-Unicode-Zeichen zum Einsatz.

Da diese Aufgaben ephemere IDs erhalten, wird der Fortschritt im Übungsmodus nicht dauerhaft gespeichert. Diese Entscheidung reflektiert die Natur von Übungen als Wegwerf-Tests. Der freie Simulator speichert jedoch weiterhin automatisch, um ein gutes Benutzererlebnis zu gewährleisten.

4.7.4 UI-Entscheidungen (Enrico Ebert)

Das UI visualisiert abstrakte Bitoperationen detailliert auf Byte-Ebene. Wir haben jeden Zwischenschritt aufklappbar gemacht, sodass Studierende selbst entscheiden können, wie tief sie in die Details einsteigen möchten.

Bei UTF-8 werden Steuerbits zur besseren Unterscheidung rot und Nutzbits blau hervorgehoben. Diese Farbcodierung half uns dabei, die komplexen Bitmuster der UTF-8-Länge-Codierung verständlicher zu machen. Bei Base64 weisen wir die 6-Bit-Gruppen und das Padding explizit aus. Nicht-druckbare Zeichen werden als Escape-Sequenzen dargestellt, um das Layout nicht zu brechen.

Wir verzichten bewusst auf zeitgesteuerte Animationen. Der statische Step-by-Step-Ansatz gibt den Studierenden die nötige Zeit, die Transformationen in ihrem eigenen Tempo zu analysieren. Animationen hätten zwar optisch ansprechender gewirkt, aber den didaktischen Wert reduziert, da Studierende nicht in der Lage sind, einzelne Schritte zu kontrollieren.

Im Übungsmodus gibt ein Validierungspanel die Lösung erst bei korrekter Eingabe frei. Das automatische Speichern ist im Übungsmodus deaktiviert und greift nur im freien Simulator-Modus, da wir es als wichtig erachteten, dass Studierende ihre Eingaben bewusst und nicht versehentlich speichern.

4.8 Java-Programmierumgebung (Markus Nöth)

Die Java-Programmierumgebung ist Bestandteil des Moduls „Programmieren 1“ und ermöglicht das Schreiben, Kompilieren und Ausführen von Java-Programmen direkt im Browser ohne Server-Backend.

4.8.1 Entscheidungen in der Architektur (Markus Nöth)

Die Umgebung folgt einer mehrschichtigen Architektur aus Präsentation, Orchestrierung und isolierter Ausführungsumgebung:

1. **Präsentationsschicht:** Der React-Kontext `JavaExecutionProvider` verwaltet den Gesamtzustand über das Datenobjekt `JavaExecutionSnapshot` (Status, Konsolenausgaben, Compiler-Diagnosen und Fortschrittsanzeige). Im Monaco-Editor [5] (`JavaEditorPanel`) begrenzt die Anwendung den Quelltext über `javaEditorLimits.ts` auf 500.000 Zeichen und 10.000 Zeilen und somit ca. 500 kB und visualisiert Fehlermeldungen direkt im Code.
2. **Orchestrierungsschicht:** Der Hook `useJavaExecution` steuert den Lebenszyklus des Web Workers (`javaRunner.worker.ts`) über asynchrone Nachrichten mit eindeutigen Request-IDs. Eine konfigurierbare Laufzeitüberwachung schützt vor Endlosschleifen: Überschreitet ein Programm das Zeitlimit, beendet der Hauptthread den Worker mittels `worker.terminate()` und startet eine neue Instanz. Dadurch bleibt die Benutzeroberfläche stets reaktionsfähig.
3. **Compiler- und Wasm-Laufzeit:** Der Worker lädt OpenJDK-`javac` und den TeaVM-Compiler [6] als statische Assets nach. Nach der Syntax- und Typprüfung durch `javac` übersetzt TeaVM den erzeugten Bytecode in Wasm, das anschließend in einer isolierten Browser-Sandbox ausgeführt wird.

Der Ablauf vom Quellcode bis zur Programmausgabe gliedert sich in drei aufeinanderfolgende Schritte:

1. **Quelltext-Vorverarbeitung:** Da Wasm im Browser keine synchron blockierenden Leseoperationen auf `System.in` unterstützt, analysiert die Funktion `inlineScannerInput()` den Quelltext vor der Übersetzung. Der Parser `ScannerInputCursor` liest die im Eingabefeld bereitgestellten Daten sequenziell aus und ersetzt Aufrufe wie `nextLine()`, `nextInt()`, `nextDouble()` oder `next()` deterministisch durch feste Literale im Syntaxbaum.
2. **Übersetzung und Sandbox-Start:** Der vorverarbeitete Quelltext wird im Web Worker durch `javac` kompiliert und via TeaVM in ein Wasm-Modul transformiert, das in der Sandbox instanziiert wird.
3. **Pufferung des Ausgabestroms:** Standardausgaben (`stdout`) und Fehlerausgaben (`stderr`) werden von TeaVM zeichenweise an Hilfsfunktionen übergeben. Ein Ausgabetimer (`createBufferedWorkerOutput`) sammelt die Zeichen und sendet sie gebündelt alle 50 Millisekunden per `postMessage` an den React-Zustand. Diese Taktung verhindert, dass hochfrequente Einzelausgaben den Hauptthread durch permanente Re-Renderings überlasten.

4.8.2 Entscheidungen im UI (Markus Nöth)

Die Arbeitsoberfläche orientiert sich an vertrauten IDEs wie Visual Studio Code:

- **Monaco-Editor:** Der Editor bietet Code-Faltung, Volltextsuche (`Strg+F`) und automatische Einrückung. Diagnosemeldungen des Compilers, beispielsweise fehlende Semikolons oder inkompatible Typen, werden als rote Inline-Marker an der betroffenen Zeile hervorgehoben.
- **Eingabebereich für `System.in`:** Über ein separates Texteingabefeld kann der Benutzer Konsoleneingaben vorab definieren, die der Vorprozessor beim Programmstart verarbeitet.
- **Konsole und Problemliste (`JavaConsolePanel`):** Der Ausgabebereich schaltet zwischen der Programmausgabe (mit optischer Trennung von regulären Ausgaben und Fehlermeldungen) und einer tabellarischen Fehlerliste um. Ein Klick auf einen Tabelleneintrag führt den Cursor direkt zur fehlerhaften Quelltextposition.

- **Steuerleiste:** Beinhaltet Schaltflächen für Start, Abbruch, Zurücksetzen auf den Ausgangszustand und den Vollbild-Fokusmodus.

4.8.3 Andere Entscheidungen (Markus Nöth)

Das Modul enthält vier interaktive Beispielprogramme und 13 strukturierte Übungsaufgaben (u. a. Konsolenausgaben, Schleifen, Variablenberechnungen). Die Aufgabenprüfung kompiliert den eingegebenen Code und führt ihn gegen definierte Testfälle aus. Bei Aufgaben mit vorgegebenen Variablenbelegungen ersetzt ein regulärer Ausdruck (`sourceTransform`) die Initialisierungswerte im Code durch alternative Testdaten, um die Allgemeingültigkeit der Lösung zu prüfen. Ändert der Lernende jedoch die Bezeichner oder den syntaktischen Aufbau dieser Vorgabezeilen tiefgreifend, greift der Ersetzungsausdruck nicht mehr. In diesem Fall kann die Validierung fehlschlagen, obwohl der geschriebene Algorithmus fachlich korrekt ist.

Für die Speicherung im Local Storage und das Teilen von Lösungen über URL-Parameter nutzt `javaCodeTransfer.ts` ein zweistufiges Verfahren: Häufige Java-Schlüsselwörter und Sonderzeichen werden über ein Wörterbuch auf Einzelbytes (0x80 bis 0xFF) komprimiert, anschließend mit dem Deflate-Algorithmus verkleinert und als Base64URL kodiert.

5 Evaluation & Projekterfahrung

5.1 Qualitätssicherung (Enrico Ebert)

Die Qualitätssicherung im Projekt folgte einem mehrstufigen Ansatz, der sowohl automatisierte als auch manuelle Prüfungen umfasste. Das Ziel war es, die Zuverlässigkeit der Plattform sicherzustellen und Fehler frühzeitig im Entwicklungsprozess zu erkennen.

5.1.1 CI/CD-Pipeline (Enrico Ebert)

Die automatisierte Pipeline wurde auf einem GitLab Runner betrieben, der auf einem Raspberry Pi gehostet wurde. Bei jedem Merge Request und Push in den Development-Branch wurde die Pipeline automatisch ausgelöst. Sie bestand aus mehreren Stufen:

- **Linting und Formatierung:** ESLint prüfte den TypeScript-Code auf stilistische Probleme und potenzielle Fehler. Prettier stellte eine einheitliche Code-Formatierung sicher.
- **Unit-Tests:** Vitest führte isolierte Komponententests durch. Besonderer Fokus lag auf mathematischen Funktionen, bei denen Genauigkeit entscheidend ist.
- **Build:** Vite erstellte einen optimierten Produktions-Build. Fehler bei der Kompilierung führten zum sofortigen Pipeline-Stopp.
- **E2E-Tests:** Playwright automatisierte Browser-Tests, die kritische Nutzerpfade wie Import/Export-Vorgänge und Simulationsabläufe abdeckten.

Nur wenn alle Stufen erfolgreich durchlaufen wurden, war ein Merge in den Development-Branch möglich. Dadurch wurden Inkonsistenzen und Regressionen frühzeitig erkannt.

5.1.2 Testing-Strategie (Enrico Ebert)

Die Teststrategie unterschied zwischen Unit-Tests und End-to-End-Tests. Unit-Tests wurden für mathematische Kernfunktionen geschrieben, da hier die größte Fehlergefahr bestand. Die Bitoperationen und mathematischen Berechnungen wurden systematisch mit bekannten Eingabe-Ausgabe-Paaren validiert.

End-to-End-Tests konzentrierten sich auf die Simulator-Workflows. Ein typischer Testablauf umfasste das Laden einer Beispielmaschine, das Ausführen einer Simulation und das Überprüfen des erwarteten Ergebnisbandes. Die Playwright-Tests wurden headless auf dem CI-Runner ausgeführt und erstellten Screenshots bei Fehlern.

5.1.3 Manuelle Prüfung (Enrico Ebert)

Neben der Automatisierung wurde das Team in regelmäßigen Reviews zusammengebracht. Merge Requests erforderten die Genehmigung eines anderen Teammitglieds. Dabei wurden neben dem Code auch architektonische Entscheidungen und die Einhaltung der Projektkonventionen geprüft.

5.1.4 Erfahrungen (Enrico Ebert)

Die CI/CD-Pipeline auf dem Raspberry Pi erwies sich als zuverlässig, allerdings mit Einschränkungen bei der Geschwindigkeit. Die ARM-Architektur führte zu längeren Build-Zeiten im Vergleich zu Cloud-basierten Runnern. Für zukünftige Projekte würde eine Hybridlösung mit schnellerem Build-Runner und lokaler Testausführung empfohlen.

Die Kombination aus automatisierten Tests und manuellem Code-Review half dabei, die KI-bedingten Fehlerquellen zu erkennen und zu korrigieren. Insbesondere Edge Cases wie NaN, Infinity und Overlong Encodings wurden durch gezielte Unit-Tests abgesichert.

5.2 Abgleich mit dem Lastenheft (Kristian Popov)

Der im Lastenheft festgelegte fachliche Umfang wurde umgesetzt und im Verlauf des Projekts erweitert: Zu den ursprünglich vorgesehenen Themen kamen die regulären Sprachen und die Kodierungsverfahren hinzu, mit Programmieren 1 außerdem ein zweiter Lernbereich. Die Erweiterungen wurden mit dem Auftraggeber abgestimmt und in die bestehende Plattform integriert (siehe Abschnitt 5.4).

Die didaktische Kernanforderung, nicht nur Ergebnisse, sondern auch Zwischenschritte sichtbar zu machen, wurde in allen Themen berücksichtigt: Simulationen lassen sich schrittweise ausführen, anhalten und zurücksetzen, und die Übungen geben begründete Rückmeldungen statt einer reinen Ja-Nein-Aussage. Auch der Betrieb ohne Backend ist umgesetzt, da Aufgabenstände im lokalen Speicher des Browsers liegen und Aufgaben über Import und Export ausgetauscht werden.

Die geforderte Erweiterbarkeit hat sich im Projektverlauf bewährt. Neue Themen wurden über den zentralen Modulkatalog registriert, ohne Navigation oder Routing der bestehenden Bereiche umbauen zu müssen. Das Unified UI-Konzept hält dabei Aufbau, Tabs und Rückmeldungen über die Themen hinweg wiedererkennbar. Dasselbe Prinzip trägt die Mehrsprachigkeit: Die Oberfläche ist vollständig übersetzt, und eine weitere Sprache wird über den zentralen Sprachkatalog und ein zusätzliches Verzeichnis mit Übersetzungsdateien ergänzt, ohne die Themen selbst anzupassen. Das Farbschema folgt wahlweise der Systemeinstellung oder einer manuellen Auswahl.

Die Qualitätsziele des Lastenhefts werden in der CI-Pipeline bei jedem Merge Request automatisch überprüft (siehe Abschnitt 5.1). Sie besteht aus sieben aufeinander aufbauenden Stufen: Prüfung der Werkzeugversionen, Bau des Java-Compilers, Formatierung und Lint, Unit-Tests, Build, E2E-Tests und Lighthouse-Messung. Jede Stufe startet erst nach einem erfolgreichen Vorgänger, und nur ein vollständig fehlerfreier Lauf gibt den Merge frei. Für das Qualitätsziel Performance wertet die letzte Stufe den Median aus drei Läufen je Seite aus. Überschreitet der LCP einer Themenseite 1,4 Sekunden, schlägt sie fehl; für die Java-Seite, die den Compiler lädt, gilt eine Schranke von 5 Sekunden. Dass die Themenseiten einzeln nachgeladen werden,

hält die je Aufruf übertragene Datenmenge klein. Ergänzend sichert die Pipeline die Herkunft der Abhängigkeiten ab, indem sie deren Signaturen prüft und nur Paketversionen mit einem Mindestalter von sieben Tagen zulässt.

Bis auf drei Punkte wurden die Anforderungen des Lastenhefts damit umgesetzt. Eine systematische Usability-Erhebung mit Studierenden war nicht möglich, da die Anwendung auf Wunsch des Auftraggebers nicht öffentlich betrieben, sondern als statischer Build ausgeliefert wird. Die Barrierefreiheit prüft die Pipeline nicht automatisch, weil die Lighthouse-Stufe bewusst auf die Kategorie Performance beschränkt ist. Messungen für kleine Bildschirme fehlen, da alle Lighthouse-Läufe mit dem Desktop-Profil erfolgen. Alle drei Punkte werden im Ausblick wieder aufgegriffen (siehe Abschnitt 6.2).

5.3 Lessons Learned (Alle)

Kristian Popov: Am meisten habe ich darüber gelernt, wie ich effizient mit KI arbeite. Entscheidend war weniger die Wahl des Modells als seine Anbindung an das Projekt: Über den Sprachserver (LSP) und die Entwicklerwerkzeuge des Browsers fragte das Modell Signaturen und Laufzeitverhalten ab, statt sie aus dem Quelltext abzuleiten. Zusammen mit einer Regeldatei für Konventionen und Architekturgrenzen brauchte es weniger Tokens, und erfundene Bezeichner traten seltener auf.

Außerdem habe ich gelernt, Aufgaben konsequent zu planen und festzuhalten. Über GitLab Issues und Milestones habe ich größere Vorhaben in überschaubare Arbeitspakete zerlegt. Gerade weil ich an zwei Themen parallel gearbeitet habe, blieb dadurch jederzeit klar, was noch offen war.

Wie viel eine CI-Pipeline abnimmt, habe ich ebenfalls erst im Projekt gemerkt. Formatierung, Lint, Tests und Build liefen bei jeder Änderung automatisch, sodass ich Fehler nicht erst am Ende einer Entwicklung gefunden habe. Gerade bei vielen kleinen Änderungen habe ich mich darauf verlassen können.

Duy Tan Pham: Im Projekt habe ich gelernt, größere Funktionen schrittweise zu entwickeln und dabei ihre Einbindung in die gesamte Plattform mitzudenken. Durch einzelne Ausbaustufen konnten neue Anforderungen aufgenommen und vorhandene Abläufe nach jeder Erweiterung erneut geprüft werden. Der Abgleich mit den anderen Themenbereichen half außerdem, die Bedienung einheitlich zu gestalten.

Bei Architekturentscheidungen zeigte sich, dass Wiederverwendung nicht automatisch mehr Abstraktion bedeutet. Eine gemeinsame Zustandsabstraktion für mehrere Eingaben wurde wieder entfernt, weil die eigenständigen Komponenten leichter nachzuvollziehen waren. Gemeinsame Seitenrahmen und Bedienmuster waren dagegen an mehreren Stellen sinnvoll. Ich habe daraus gelernt, eine Abstraktion nur dann einzuführen, wenn sie ein tatsächlich wiederkehrendes Problem vereinfacht.

Eine weitere Erkenntnis war, dass nicht jede technisch mögliche Erweiterung das Produkt verbessert. Einige Ansätze wirkten zunächst sinnvoll, passten nach genauerer Prüfung aber nicht zu den Anforderungen oder erhöhten die Komplexität unnötig. Deshalb war es wichtig, Entscheidungen erneut zu hinterfragen und Lösungen bei Bedarf bewusst zurückzunehmen. Dadurch blieb der Fokus auf einem verständlichen und verlässlichen Lernerlebnis.

Richard Schröder: Für die Zusammenarbeit im Team war ein einheitliches Verständnis der Anforderungen entscheidend. Wenn vor der Umsetzung nicht genau geklärt war, wie eine Komponente aufgebaut sein soll, wichen die Teilergebnisse voneinander ab und mussten nachträglich mühsam angeglichen werden.

Beim Einsatz von KI-Werkzeugen zeigte sich, dass der Zeitgewinn durch den hohen Aufwand für Prompt-Zerlegung, Fehlersuche sowie das Refactoring von totem oder doppeltem Code oft geringer ausfiel als erwartet. Da Modelle dazu neigten, Probleme unnötig zu verkomplizieren oder Änderungen an falschen Stellen im Projekt vorzunehmen, blieben gründliche manuelle Reviews und Bereinigungen notwendig. Zudem fiel der eigene Lerneffekt beim reinen Übernehmen generierter Lösungen geringer aus als beim eigenständigen Erarbeiten der Lösungswege.

Auch bei der Versionsverwaltung mit Git zeigte sich die Bedeutung enger Absprachen: Wurden Branches über längere Zeit ohne Zwischenabgleich entwickelt, führten gleichzeitige Änderungen an denselben Dateien schnell zu unübersichtlichen Merge-Konflikten. Kleinere und regelmäßig zusammengeführte Stände verhinderten diesen Mehraufwand.

Enrico Ebert: Bei der Arbeit mit BigInt wurde mir deutlich, dass JavaScripts mathematische Standardtypen bei großen Ganzzahlen an Präzision verlieren. Die Konvertierung zwischen verschiedenen Basen musste vollständig selbst implementiert werden, da keine native Funktion für Basis-zu-Basis-Umrechnung existiert. Die Visualisierung von IEEE-754 erwies sich als didaktisch besonders wertvoll: Studierende, die zuvor den Zusammenhang zwischen Dezimalzahl und Bitrepräsentation nicht verstanden, konnten durch die schrittweise Aufschlüsselung den Normalisierungsprozess nachvollziehen.

Die Validierung von UTF-8-Bytesströmen war komplexer als anfänglich angenommen. Insbesondere die korrekte Behandlung von Overlong Encodings erforderte intensives Lesen der Unicode-Spezifikation. Die Sicherheitsrelevanz dieser Prüfungen wurde mir erst durch die Auseinandersetzung mit Web-Sicherheitsthemen wie dem Facebook-Encoding-Bug bewusst. Die programmatische Übungsgenerierung erwies sich dagegen als besonders wertvoll: Sie bietet unbegrenzte Variationsmöglichkeiten ohne eine Datenbank pflegen zu müssen. Der Schwierigkeitsgrad wird über Parameter gesteuert und kann leicht angepasst werden.

Markus Nöth: Bei der Arbeit an den übergreifenden Plattformkomponenten habe ich gelernt, wie wichtig eine saubere Branching-Strategie und klare Schnittstellen im Team sind. Da zentrale Teile wie der Modulkatalog, die Zustandsgraph-Komponente oder die Transfer-Codecs von mehreren Teammitgliedern gleichzeitig genutzt wurden, verhinderten nur kurzlebige Feature-Branches und regelmäßige Absprachen größere Konflikte beim Zusammenführen.

Beim Einsatz von KI-Werkzeugen habe ich die Erfahrung gemacht, dass Sprachmodelle zwar schnell lauffähige Codefragmente erzeugen, dabei aber oft zu unnötig komplizierten Architekturen neigen oder Änderungen an falschen Stellen vorschlagen. Ohne den in Kapitel 3.1 beschriebenen Prozess aus eigener Konzeption, schrittweiser Aufforderung und manueller Gegenprüfung verliert man schnell den Überblick über den generierten Code. Die automatisierte CI-Pipeline war eine große Hilfe: Durch Linting und automatische Kompilierungstests erhielten wir direkt beim Push Rückmeldung, ob Fehler entstanden waren, bevor fehlerhafte Stände im gemeinsamen Repository landeten.

Ebenso wichtig war für mich die Arbeit mit den GitLab-Work-Items. Durch das Festhalten und stetige Aktualisieren der Aufgaben blieb jederzeit sichtbar, welche Punkte noch offen waren, sodass keine To-dos verloren gingen. Über die Kommentarfunktion ließen sich gefundene Zwischenergebnisse und getroffene Entscheidungen direkt dokumentieren. Ein konkretes Beispiel war das geplante Deployment über GitLab Pages: Als sich im Austausch mit dem Auftraggeber und der Hochschulinfrastruktur herausstellte, dass dieser Weg so nicht umgesetzt werden konnte, habe ich die Begründung und das alternative Vorgehen direkt als Kommentar im Ticket hinterlegt. So wussten alle Teammitglieder sofort Bescheid, ohne dass Informationen in Chat-Verläufen untergingen.

5.4 Design-Iterationen (Duy Tan Pham)

Die Anforderungen entwickelten sich während des Projekts weiter. Den fachlichen und technischen Ausgangspunkt bildeten die im Lastenheft festgehaltenen Ziele. Rückmeldungen des Professors, neue fachliche Wünsche und Erfahrungen mit den ersten Prototypen führten im weiteren Verlauf zu Anpassungen. Die grundlegenden Ziele blieben erhalten. Dazu gehören das selbstständige Lernen, eine einheitliche Bedienung und der Betrieb ohne Backend. Der Umfang und die technische Umsetzung entwickelten sich dagegen schrittweise weiter.

Der anfangs vorgesehene fachliche Umfang konzentrierte sich auf Themen aus Grundlagen Informatik. Später wuchs er auf zwei Ebenen. Zum einen kamen mit den regulären Sprachen und den Kodierungsverfahren zwei weitere Fachthemen hinzu. Zum anderen wurden vorhandene Funktionen weiter konkretisiert. Im DNF-Bereich wurde das im Lastenheft vorgesehene Vereinfachen als eigener interaktiver Arbeitsschritt mit Prüfung und Lösungsweg umgesetzt. Mit Programmieren 1 und der Java-Umgebung kam später ein ergänzender Lernbereich außerhalb von Grundlagen Informatik hinzu. Die Erweiterungen wurden jeweils vorher abgestimmt. Für die Plattform bedeutete das, dass die gemeinsame Struktur mehrmals neue Themen aufnehmen musste, ohne die bestehenden Bereiche dafür umzubauen.

Die grobe Seitenarchitektur wurde bereits zu Beginn des Projekts festgelegt. Schon das frühe Plattformgerüst registrierte Lernbereiche, Themenbereiche, Routen und Darstellungsinformationen zentral in `moduleCatalog.tsx`. Mit `ModuleTopicPage` stand außerdem ein gemeinsamer Aufbau für die Themenseiten bereit. Dadurch konnten die Fachmodule auf einer einheitlichen Grundlage entwickelt werden. Im weiteren Verlauf wurden diese Bausteine anhand der konkreten Anforderungen der Module erweitert und präzisiert.

Trotz dieser gemeinsamen Grundlage unterschieden sich die parallel entwickelten Themen zunächst im konkreten Aufbau ihrer Übersichten, Beispiele und Übungen. Diese Bereiche wurden während der Integration schrittweise vereinheitlicht. Gemeinsame Tabs, Aufgabenkomponenten und Simulationsrahmen sorgen heute für einen wiedererkennbaren Ablauf. Gleichzeitig bleibt die Fachlogik in den jeweiligen Themenbereichen. Diese Aufteilung wurde der vollständigen Zentralisierung vorgezogen, weil sich die fachlichen Modelle und Prüfverfahren deutlich unterscheiden.

Bei Endlichen Automaten und Regulären Sprachen war das besonders deutlich. Beide Themen wurden zunächst unabhängig umgesetzt. Bei der Integration wurden dann gemeinsame Grammatik- und Automatenmodelle sowie Bausteine für das Übertragungsformat wiederverwendet. Die themenspezifischen Prüfverfahren blieben dagegen in den jeweiligen Bereichen.

Eine weitere Änderung betraf Import, Export und die Speicherung von Aufgabenständen. Import und Export waren anfangs nur als optionale Funktionen vorgesehen und wurden zuerst in einzelnen Simulatoren umgesetzt. Später wurden das Datenformat und der Einstieg über den globalen Kopfbereich zusammengeführt. Daraus entstand die gemeinsame Training-Center-Struktur. Zusätzlich werden geeignete Aufgabenstände und Einstellungen im lokalen Speicher des Browsers gesichert. Damit bleiben Ergebnisse zwischen Aufrufen erhalten, ohne die ursprüngliche Entscheidung gegen Benutzerkonten und ein Backend aufzugeben.

Die gemeinsame Struktur entstand aus den wiederkehrenden Abläufen der Fachthemen und wurde angepasst, sobald mehrere unabhängige Lösungen dieselbe Aufgabe erfüllten. Dadurch konnten neue Anforderungen aufgenommen werden, ohne die Plattform in voneinander getrennte Werkzeuge aufzuteilen.

6 Fazit & Ausblick

6.1 Zusammenfassung der Projektergebnisse (Alle)

Kristian Popov: Mein Schwerpunkt im Projekt lag auf den Themen *Endliche Automaten* und *Reguläre Sprachen*. Für die Endlichen Automaten ist eine Umgebung entstanden, in der Studierende deterministische und nichtdeterministische Automaten definieren und Wörter Schritt für Schritt simulieren; beide Automatentypen teilen sich eine gemeinsame Ausführungslogik (siehe Abschnitt 4.2). Das Thema *Reguläre Sprachen* verbindet Sprachbeschreibung, Grammatik und Worterkennung über dieselbe Fachlogik, sodass Aufzählung, Worterkennung und Übungskontrolle stets übereinstimmen (siehe Abschnitt 4.3).

Beide Themen bewerten Übungen über das Verhalten der Lösung statt über ihre Struktur. Studierende erhalten dadurch auch für eigenständig aufgebaute Automaten und Grammatiken eine Rückmeldung mit konkretem Gegenbeispiel statt einer reinen Ja-Nein-Aussage.

Bei beiden Themen war mir wichtig, nicht nur Ergebnisse zu zeigen, sondern auch den Weg dorthin. Übergangstabelle und Zustandsdiagramm sind an denselben Simulationsschritt gebunden, und der Stapeltest macht sichtbar, ob ein Automat die beabsichtigte Sprache erkennt und nicht nur das gerade ausprobierte Wort. Die Übungskontrolle vergleicht das Verhalten statt der Struktur einer Lösung und gibt bei Fehlern konkrete Gegenbeispiele aus.

Zusätzlich habe ich das Zustandsdiagramm der Endlichen Automaten auf die gemeinsame Graphkomponente umgestellt, die auch die Moore- und Mealy-Automaten verwenden, und die Automaten- und Grammatikfunktionen von der Oberfläche getrennt gehalten. Dadurch konnten die *Regulären Sprachen* sie unverändert weiterverwenden. Automaten, Grammatiken und Sprachbeschreibungen lassen sich über die gemeinsame Import- und Exportfunktion weitergeben, und der Bearbeitungsstand einer Übung bleibt beim Wechsel erhalten.

Duy Tan Pham: Mein fachlicher Schwerpunkt lag auf dem Themenbereich *Disjunktive Normalform*. Dort können Studierende boolesche Ausdrücke und Wahrheitstabellen untersuchen, die kanonische DNF schrittweise herleiten und Ausdrücke vereinfachen. Beispiele zeigen einen vollständigen Lösungsweg. In den Übungen bearbeiten die Lernenden dagegen Wahrheitstabellen oder DNF-Ausdrücke selbst und erhalten eine fachlich berechnete Rückmeldung. Dabei werden auch logisch gleichwertige Schreibweisen erkannt.

Die Minimierung liefert eine DNF mit minimaler Literalzahl. Zusätzlich kann für bis zu vier Variablen eine global kleinste Formel aus UND- und ODER-Verknüpfungen bestimmt werden. Die Fachlogik ist von der Benutzeroberfläche getrennt und durch automatisierte Tests abgesichert. Der DNF-Bereich nutzt außerdem den gemeinsamen Themenaufbau sowie die vorhandenen Funktionen für Import und Export. Dadurch fügt er sich in Bedienung und Darstellung in die gesamte Lernplattform ein.

Richard Schröder: Mein fachlicher Schwerpunkt lag auf dem Themenbereich *Turingmaschine*. Dafür habe ich eine interaktive Simulationsumgebung entwickelt, in der Maschinen über eine tabellenbasierte Oberfläche konfiguriert, Eingabebänder definiert und die Einzelschritte vorwärts sowie rückwärts ausgeführt werden können. Die Banddarstellung passt sich an die Bildschirmbreite an und visualisiert die Kopfposition samt Übergangsstatus. Neben einer freien Simulation umfasst das Modul vorgefertigte Beispiele sowie Übungsaufgaben, bei denen erstellte Maschinen gegen mehrere automatisierte Testfälle auf korrekte Bandinhalte und gültige Haltezustände geprüft werden.

Auf Architekturebene habe ich das generische Interface `ISimulator<TConfig, TState>` sowie den zugehörigen Steuerungs-Hook entwickelt, um die mathematische Logik von der UI zu tren-

nen und eine einheitliche Basis für alle Simulatoren der Plattform bereitzustellen. Ein weiterer technischer Schwerpunkt war der Transfer-Codec für den Import, Export und die persistente Speicherung: Dafür habe ich ein eigenes, speziell auf Turingmaschinen optimiertes Kompressionsverfahren auf Bitstream-Basis entworfen, das Zustände, Alphabete und Übergangstabellen kompakt als Base64URL-Zeichenkette kodiert.

Enrico Ebert: Mein fachlicher Schwerpunkt lag auf den Themenbereichen *Zahlensysteme* und *Encoding*. Im Bereich Zahlensysteme habe ich Simulatoren für die Umrechnung zwischen verschiedenen Basen, Rechenoperationen mit negativen Zahlen im Zweierkomplement sowie eine detaillierte Visualisierung von IEEE-754-Gleitkommazahlen entwickelt. Die Verwendung von BigInt stellte die korrekte Verarbeitung großer Zahlen sicher. Die schrittweise Darstellung der Bitrepräsentation ermöglichte es Studierenden, den Normalisierungsprozess und die Gleitkommadarstellung nachzuvollziehen.

Im Encoding-Modul wurden die Transformationen zwischen Text, Codepoints und Byte-Sequenzen für UTF-8 und Base64 implementiert. Die visuelle Aufschlüsselung der Bitoperationen, einschließlich der Unterscheidung von Steuerbits und Nutzbits, hilft Studierenden die Funktionsweise dieser allgegenwärtigen Standards zu verstehen. Die programmatische Generierung von Übungsaufgaben bot unbegrenzte Variationsmöglichkeiten ohne statische Aufgabenlisten pflegen zu müssen.

Die Tests für die mathematischen Module wurden auf die Kernfunktionen konzentriert. BigInt-Operationen und IEEE-754-Bitmanipulationen wurden systematisch mit bekannten Eingabe-Ausgabe-Paaren validiert. Die UTF-8-Dekodierung und Base64-Transformation wurden durch gezielte Unit-Tests abgesichert. Dies hilft Studierenden, mit verlässlichen Simulationen zu arbeiten, die korrekte Ergebnisse liefern.

Markus Nöth: Mein fachlicher Schwerpunkt im Projekt lag auf der Umsetzung der *Java-Programmierumgebung* sowie des Moduls für *Moore- und Mealy-Automaten*.

Für die Java-Umgebung war es mir wichtig, eine Java-Lernumgebung ohne Anwendungsbackend direkt im Browser bereitzustellen. Ich habe dafür den Monaco-Editor eingebunden, der Studierenden durch Fehler-Marker und bekannte Tastenkürzel eine vertraute Arbeitsweise ermöglicht. Die Kompilierung mit OpenJDK-`javac` und die Ausführung via TeaVM in Wasm laufen getrennt von der Benutzeroberfläche in einem Compiler- und einem Runner-Worker ab. Ausgaben werden über einen Puffer gebündelt in die Konsole gestreamt, und für Konsoleneingaben habe ich eine Quelltext-Vorverarbeitung geschrieben, die `Scanner`-Aufrufe vorab durch Eingabewerte ersetzt.

Bei den Moore- und Mealy-Automaten habe ich die Simulationslogik beider Typen auf Basis des gemeinsamen Interfaces `ISimulator<TConfig,TState>` umgesetzt. Dabei lag mir besonders am Herzen, möglichst viele Komponenten als wiederverwendbare Bausteine unter `components/shared/` auszulegen. Sowohl die interaktive Graphdarstellung `StateMachineGraph` mit radialer Knotenverteilung als auch das Logikgatter-Diagramm `LogicGateDiagram` wurden so konzipiert, dass auch andere Module der Plattform sie nutzen können. Zudem habe ich für beide Automatentypen die Umwandlung in Wahrheitstabellen, disjunktive Normalformen (DNF) und deren Minimierung innerhalb der beschriebenen Variablengrenzen integriert.

Über diese beiden Fachmodule hinaus habe ich die übergeordnete modulare Architektur der Plattform entworfen. Dazu gehörten das Seitenlayout, das asynchrone Code-Splitting über den zentralen Modulkatalog sowie die Koordination der internen Kommunikation, der Aufgabenverteilung und der regelmäßigen Abstimmungstermine mit dem Auftraggeber.

6.2 Perspektiven (Kristian Popov)

Ein naheliegender nächster Schritt ist die Erweiterung um weitere Lernbereiche – beispielsweise um zusätzliche Themen, die von Professoren beigesteuert oder definiert werden. Da Lernbereiche und Themen zentral im Modulkatalog registriert sind, werden Navigation und Routing daraus dynamisch abgeleitet. Ein neuer Lernbereich benötigt lediglich eine eigene Übersichtsseite, während ein neues Thema zusätzlich seine Fachlogik und einen eigenen Simulator mitbringt.

Auch die Erstellung von Aufgaben könnte stärker über die Oberfläche erfolgen. Lehrende könnten Aufgaben und Beispiele direkt anlegen und weitergeben, wobei das bestehende Übertragungsformat um Angaben wie Beschreibung, Schwierigkeitsgrad und Prüfkriterien erweitert werden könnte.

Darauf aufbauend wäre eine automatische Generierung von Aufgabenvarianten möglich. Bisher sind die Varianten außerhalb der Anwendung mit KI-Unterstützung entstanden; aus einer Vorlage könnten sie künftig direkt erzeugt und anschließend automatisch überprüft werden. Ergänzend könnten Quizfragen, Lernkarten und Theoriefragen das praktische Üben erweitern.

Für Programmieraufgaben in Java wäre eine Zeilensperrung im Editor hilfreich. Marker im Quelltext würden den bearbeitbaren Bereich abgrenzen, alle übrigen Zeilen blieben schreibgeschützt. Die Machbarkeit ist geprüft; Aufgaben ließen sich damit zuverlässig automatisch überprüfen.

Für Lehrende wäre außerdem eine Auswertung der Bearbeitungsergebnisse sinnvoll. Sie könnte zeigen, welche Aufgaben besonders häufig falsch gelöst oder abgebrochen werden. Die Fortschritte könnten zusätzlich für Gamification, beispielsweise durch Streaks oder Abzeichen, genutzt werden.

Weitere Möglichkeiten sind ein PDF- oder Druckexport sowie eine verbesserte mobile Darstellung. Besonders Übergangstabellen und Zustandsdiagramme sind auf kleinen Bildschirmen derzeit noch schwer zu bedienen.

Offen bleiben außerdem drei Punkte aus dem Abgleich mit dem Lastenheft (siehe Abschnitt 5.2). Eine systematische Usability-Erhebung mit Studierenden konnte nicht stattfinden, da die Anwendung nicht öffentlich betrieben wird. Über einen hochschulintern erreichbaren Testzugang ließen sich Bedienbarkeit und Verständlichkeit der Simulatoren strukturiert erheben und die Ergebnisse gezielt in die Oberfläche zurückführen.

Ebenso ließe sich die Barrierefreiheit automatisiert absichern. Die Lighthouse-Stufe der Pipeline ist bewusst auf die Kategorie Performance beschränkt; eine zusätzliche Prüfung der Kategorie Barrierefreiheit würde Tastaturbedienung, Kontraste und Beschriftungen bei jedem Merge Request messbar machen. Ergänzende Läufe mit dem Mobilprofil würden zugleich die bislang fehlenden Messwerte für kleine Bildschirme liefern.

Insgesamt lassen sich weitere Lernbereiche, Aufgabenvarianten und Exporte vergleichsweise einfach in die bestehende Struktur integrieren. Funktionen wie Benutzerkonten, zentrale Auswertungen oder eine Anbindung an ein LMS wie Moodle würden dagegen ein Backend und serverseitige Speicherung erfordern, da die aktuelle Anwendung bewusst ohne diese Infrastruktur auskommt.

Anhang

Abkürzungsverzeichnis

ARM Advanced RISC Machine; Architektur für Mikroprozessoren, die auf Energieeffizienz ausgelegt ist

ASCII American Standard Code for Information Interchange; Amerikanischer Standard für die numerische Darstellung von Text

Base64 Base-64 Encoding; Kodierungsverfahren, das binäre Daten als ASCII-Text darstellt

BigInt Big Integer; JavaScript-Typ für ganzzahlige Werte beliebiger Größe ohne Präzisionsverlust

CD Continuous Delivery; Automatisierte Auslieferung der geprüften Anwendung

CI Continuous Integration; Automatisiertes Bauen und Prüfen bei jeder Änderung im Repository

CSS Cascading Style Sheets; Sprache, die Aussehen und Layout von Webseiten beschreibt

DFA Deterministischer endlicher Automat; Endlicher Automat, bei dem zu jedem Zustand und Eingabezeichen höchstens ein Folgezustand existiert

DNF Disjunktive Normalform; Normalform einer booleschen Funktion als ODER-Verknüpfung von UND-Termen [4, S. 163]

E2E End-to-End; Testverfahren, das die gesamte Anwendung von der Benutzeroberfläche bis zur Logik als Gesamtsystem prüft

i18n Internationalisierung; Strukturierung von Software zur Unterstützung verschiedener Sprachen und regionaler Besonderheiten

ID Identifier; Eindeutiger Bezeichner zur Identifikation von Entitäten oder Datensätzen

IDE Integrated Development Environment; Integrierte Entwicklungsumgebung, die Werkzeuge für Softwareentwicklung wie Editor, Compiler und Debugger bündelt

IEEE-754 Institute of Electrical and Electronics Engineers 754; IEEE-Standard für die Darstellung von Gleitkommazahlen in binärer Form

JSON JavaScript Object Notation; Textbasiertes Datenformat für den Austausch strukturierter Daten

JVM Java Virtual Machine; Laufzeitumgebung, die kompilierten Java-Bytecode ausführt

KI Künstliche Intelligenz; Sammelbegriff für Verfahren, die Aufgaben übernehmen, welche sonst menschliche Intelligenz erfordern; in diesem Bericht vor allem große Sprachmodelle

LCP Largest Contentful Paint; Zeitpunkt, zu dem das größte sichtbare Inhaltselement einer Seite dargestellt ist; Kennzahl für die wahrgenommene Ladezeit

LMS Learning Management System; Lernplattform zur Verwaltung von Kursen und Lernmaterialien, etwa Moodle oder ILIAS

LSP Language Server Protocol; Protokoll zwischen Editor und Sprachserver für Codenavigation, Typinformationen und Refactoring

MCP Model Context Protocol; Offener Standard, über den Sprachmodelle externe Werkzeuge und Datenquellen anbinden

NaN Not a Number; Spezieller Gleitkommawert, der das Ergebnis einer ungültigen mathematischen Operation darstellt

NFA Nichtdeterministischer endlicher Automat; Endlicher Automat, bei dem zu einem Zustand und Eingabezeichen mehrere Folgezustände möglich sind

PDF Portable Document Format; Plattformunabhängiges Dateiformat für seitengetreue Dokumente

PoC Proof of Concept; Frühzeitige Machbarkeitsstudie oder Prototyp zum Nachweis der Funktionsfähigkeit eines Konzepts

SVG Scalable Vector Graphics; XML-basiertes Dateiformat für zweidimensionale Vektorgrafiken

THWS Technische Hochschule Würzburg-Schweinfurt; Hochschule aus Würzburg und Schweinfurt (Bayern) [7]

UI User Interface; Benutzeroberfläche einer Anwendung, über die Eingaben erfolgen und Ergebnisse dargestellt werden

Unicode Unicode Standard; Internationaler Standard für die eindeutige Zuordnung von Zeichen zu Codepoints

URL Uniform Resource Locator; Adresse, über die eine Ressource im Web eindeutig bezeichnet und abgerufen wird

UTF-8 Unicode Transformation Format 8; Zeichenkodierung für Unicode-Text, die eine variable Bytelänge verwendet

Wasm WebAssembly; Binäres Instruktionsformat zur schnellen, plattformunabhängigen Codeausführung in Web-Browsern

Literatur

[1] S. Shamsi. „Graphify - Multi-modal Knowledge Graph Builder,“ besucht am 31. Juli 2026. Adresse: `https://graphify.net/`.

[2] Sakana AI. „Sakana Fugu - Multi-agent System as A Model,“ besucht am 31. Juli 2026. Adresse: `https://sakana.ai/fugu/`.

[3] G. Vossen und K.-U. Witt, *Grundlagen der Theoretischen Informatik mit Anwendungen: Eine Einführung für Studierende der Informatik, Wirtschaftsinformatik und Technischen Informatik* (Lehrbuch Informatik). Vieweg+Teubner Verlag Wiesbaden, 2000. DOI: `10.1007/978-3-322-96838-8`.

[4] W. Schiffmann und R. Schmitz, *Technische Informatik 1: Grundlagen der digitalen Elektronik*, 5. Aufl. Berlin, Heidelberg: Springer, 2003, Abschnitt 4.2.3, Quine-McCluskey-Verfahren ab S. 172, ISBN: 978-3-540-40418-7. besucht am 17. Aug. 2026. Adresse: `https://www.fernuni-hagen.de/mi/fakultaet/emeriti/docs/schiffmann/ti1.pdf`.

[5] Microsoft. „Monaco Editor - A browser-based code editor,“ besucht am 15. Aug. 2026. Adresse: `https://microsoft.github.io/monaco-editor/`.

[6] A. Andreev. „TeaVM - Ahead-of-time compiler for Java bytecode,“ besucht am 15. Aug. 2026. Adresse: `https://teavm.org/`.

[7] Technische Hochschule Würzburg-Schweinfurt. „THWS - Technische Hochschule Würzburg-Schweinfurt,“ besucht am 14. Juni 2026. Adresse: `https://thws.de`.